

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное

образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет «Систем управления и робототехники»

Лабораторная работа №3

по дисциплине

«Теория автоматического управления»

Вариант 15

Выполнил:

Скалиуш Артём (409567)

Преподаватель:

Пашенко Артем Витальевич



Санкт-Петербург 2026

Содержание

Задание 1. Синтез регулятора с заданной степенью устойчивости	3
1) Собственные числа и управляемость каждой моды	3
2) Достижимая степень устойчивости	4
3) Модель замкнутой системы	4
4) Выбор значений α	5
5) Синтез при помощи линейного матричного неравенства	5
Для $\alpha_1 = 1$	6
Для $\alpha_2 = 2$	7
6) Синтез при помощи матричного уравнения типа Риккати	9
Для $\alpha_1 = 1$	10
Для $\alpha_2 = 2$	11
7) Сопоставление результатов	13
Задание 2. Управление по выходу с заданной степенью устойчивости	14
1) Собственные числа, управляемость и наблюдаемость	14
2) Достижимые степени устойчивости и сходимости	15
3) Наборы (α_K, α_L)	15
4) Синтез K и L по LMI	15
5) Проверка для выбранных наборов	16
6) Моделирование	17
7) Сопоставление наборов (α_K, α_L)	27
Задание 3. Регулятор с качественной экспоненциальной устойчивостью	28
1) Выбор параметров β и r	28
2) Синтез для наборов (Q, R)	28
3) Проверка на комплексной плоскости	30
4) Моделирование	31
Вывод	34
Общий вывод	34
Приложение	35
Код для задания 1	35
Код для задания 2	40
Код для задания 3	46

Задание 1. Синтез регулятора с заданной степенью устойчивости

Для варианта 15 в Задании 1 используется набор №10:

$$A = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 0 & 2 \\ -2 & -2 & -4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 \\ -2 \\ 2 \end{bmatrix}.$$

Рассматривается система

$$\dot{x} = Ax + Bu, \quad u = Kx.$$

В данном задании требуется:

- определить управляемость системы и максимально достижимую степень устойчивости;
- выбрать несколько допустимых значений α ;
- для каждого выбранного α синтезировать регуляторы K_1, K_2 методом линейного матричного неравенства и регуляторы K_3, K_4 методом матричного уравнения типа Риккати;
- сравнить собственные числа матриц $A + BK$, а также результаты моделирования по управлению $u(t)$ и по состоянию $x(t)$.

1) Собственные числа и управляемость каждой моды

Собственные числа матрицы A :

$$\lambda_1 = 1 + i, \quad \lambda_2 = 1 - i, \quad \lambda_3 = -2.$$

Матрица управляемости по Калману:

$$AB = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 0 & 2 \\ -2 & -2 & -4 \end{bmatrix} \begin{bmatrix} 2 \\ -2 \\ 2 \end{bmatrix} = \begin{bmatrix} 18 \\ 8 \\ -8 \end{bmatrix}.$$

$$A^2B = A(AB) = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 0 & 2 \\ -2 & -2 & -4 \end{bmatrix} \begin{bmatrix} 18 \\ 8 \\ -8 \end{bmatrix} = \begin{bmatrix} 32 \\ 20 \\ -20 \end{bmatrix}.$$

$$U = [B, AB, A^2B] = \begin{bmatrix} 2 & 18 & 32 \\ -2 & 8 & 20 \\ 2 & -8 & -20 \end{bmatrix}.$$

$$\det(U) = 0, \quad \text{rank}(U) = 2 < 3.$$

Следовательно, система не является полностью управляемой.

Проверим каждое собственное число по Хаутусу,

$$H(\lambda_i) = [\lambda_i I - A, B].$$

Для $\lambda_1 = 1 + i$:

$$H(\lambda_1) = \begin{bmatrix} -3 + i & -1 & -6 & 2 \\ -2 & 1 + i & -2 & -2 \\ 2 & 2 & 5 + i & 2 \end{bmatrix}, \quad \text{rank}(H(\lambda_1)) = 3.$$

Для $\lambda_2 = 1 - i$:

$$H(\lambda_2) = \begin{bmatrix} -3-i & -1 & -6 & 2 \\ -2 & 1-i & -2 & -2 \\ 2 & 2 & 5-i & 2 \end{bmatrix}, \quad \text{rank}(H(\lambda_2)) = 3.$$

Для $\lambda_3 = -2$:

$$H(\lambda_3) = \begin{bmatrix} -6 & -1 & -6 & 2 \\ -2 & -2 & -2 & -2 \\ 2 & 2 & 2 & 2 \end{bmatrix}, \quad \text{rank}(H(\lambda_3)) = 2.$$

Следовательно, неуправляемой является только мода $\lambda = -2$. Она устойчива, поэтому система не полностью управляема, но стабилизируема.

2) Достижимая степень устойчивости

Степень устойчивости α означает, что для замкнутой системы должно выполняться условие

$$\Re \lambda_j(A + BK) \leq -\alpha.$$

Неуправляемое собственное число $\lambda = -2$ обратной связью изменить нельзя, поэтому оно обязательно остается в спектре матрицы $A + BK$.

Отсюда сразу следует ограничение:

- если $\alpha > 2$, то требование $\Re \lambda \leq -\alpha$ для моды $\lambda = -2$ невыполнимо;
- если $0 < \alpha \leq 2$, то это ограничение не нарушается.

Следовательно, максимальная достижимая степень устойчивости равна

$$\alpha_{\max} = 2.$$

3) Модель замкнутой системы

Модель замкнутой системы имеет вид

$$\dot{x} = (A + BK)x, \quad u(t) = Kx(t).$$

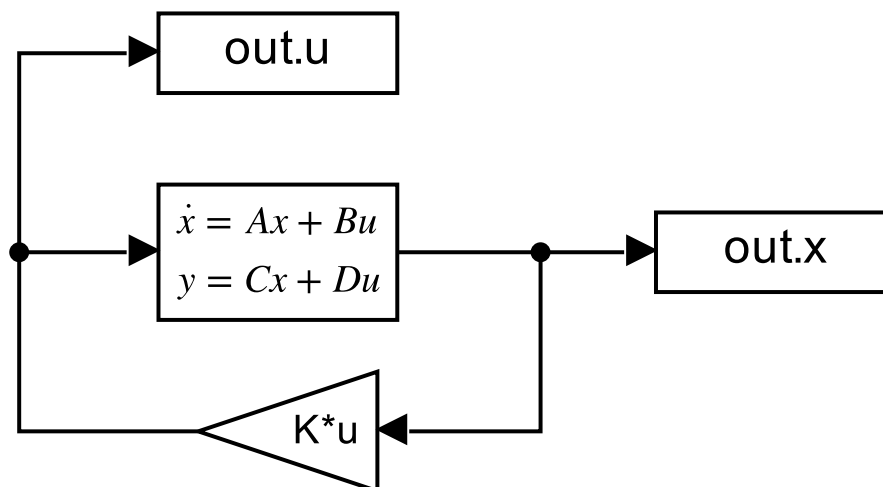


Рисунок 1. Структурная схема моделирования для задания 1

Parameters	
A:	
A	<3x3 double>
B:	
B	[2;-2;2]
C:	
eye(3)	<3x3 double>
D:	
zeros(3,1)	[0;0;0]
Initial conditions:	
[1;1;1]	

Рисунок 2. Параметры блоков модели для задания 1

Во всех численных экспериментах используются начальные условия

$$x(0) = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}^T.$$

4) Выбор значений α

Выберем два значения степени устойчивости:

$$\alpha_1 = 1, \quad \alpha_2 = 2.$$

Значение $\alpha_2 = 2$ является максимально достижимым, а $\alpha_1 = 1$ лежит внутри допустимого диапазона и заметно отличается от него.

5) Синтез при помощи линейного матричного неравенства

Используем матричное неравенство

$$AP + PA^T + 2\alpha P + BY + Y^T B^T \leq 0, \quad K = YP^{-1}, \quad P > 0.$$

Здесь матрица P задает квадратичную функцию Ляпунова, а матрица Y вводится как вспомогательная переменная, чтобы задача оставалась линейной по неизвестным. После нахождения P и Y матрица регулятора восстанавливается по формуле $K = YP^{-1}$.

В варианте K_1 требуется только найти любое допустимое решение неравенства.

В варианте K_2 дополнительно решается задача минимизации. В качестве выпуклого критерия используем минимизацию нормы Y :

$$\min \|Y\|_F.$$

Такой выбор не меняет саму идею синтеза, а лишь выбирает из множества допустимых решений более «мягкий» вариант. На практике это обычно уменьшает коэффициенты

регулятора и начальный бросок управления, при этом требуемая степень устойчивости сохраняется не хуже заданной.

Для $\alpha_1 = 1$

Получено допустимое решение без дополнительной минимизации:

$$K_1 = [-6.256187 \quad -0.600273 \quad 1.791598].$$

Собственные числа матрицы $A + BK_1$:

$$\sigma(A + BK_1) = \{-2.864316 + 10.492128i, -2.864316 - 10.492128i, -2\}.$$

Все вещественные части не превосходят -1 , поэтому требуемая степень устойчивости обеспечена.

Для варианта с минимизацией:

$$K_2 = [-1.197723 \quad -0.393918 \quad -1.197288].$$

Собственные числа матрицы $A + BK_2$:

$$\sigma(A + BK_2) = \{-1.001094 + 2.850525i, -1.001094 - 2.850525i, -2\}.$$

Здесь собственные числа расположены существенно ближе к границе $\Re \lambda = -1$, то есть замкнутая система получается менее «быстрой» и требует меньших усилий управления.

Начальные значения управления:

$$u_1(0) = K_1 x(0) = -5.064862, \quad u_2(0) = K_2 x(0) = -2.788929.$$

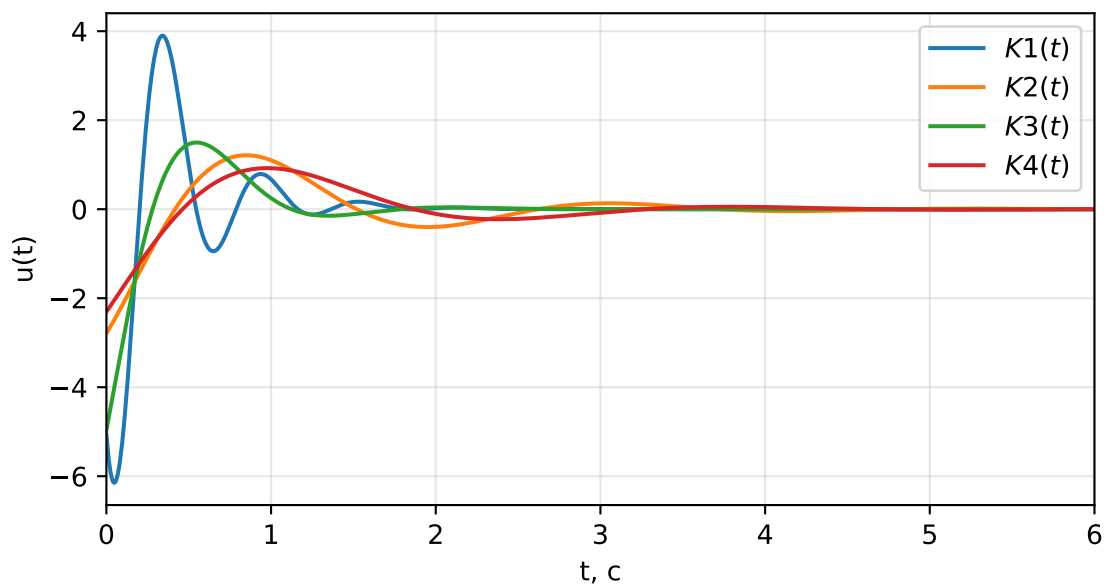


Рисунок 3. Управление $u(t)$ для $\alpha_1 = 1$ и регуляторов K_1, K_2, K_3, K_4

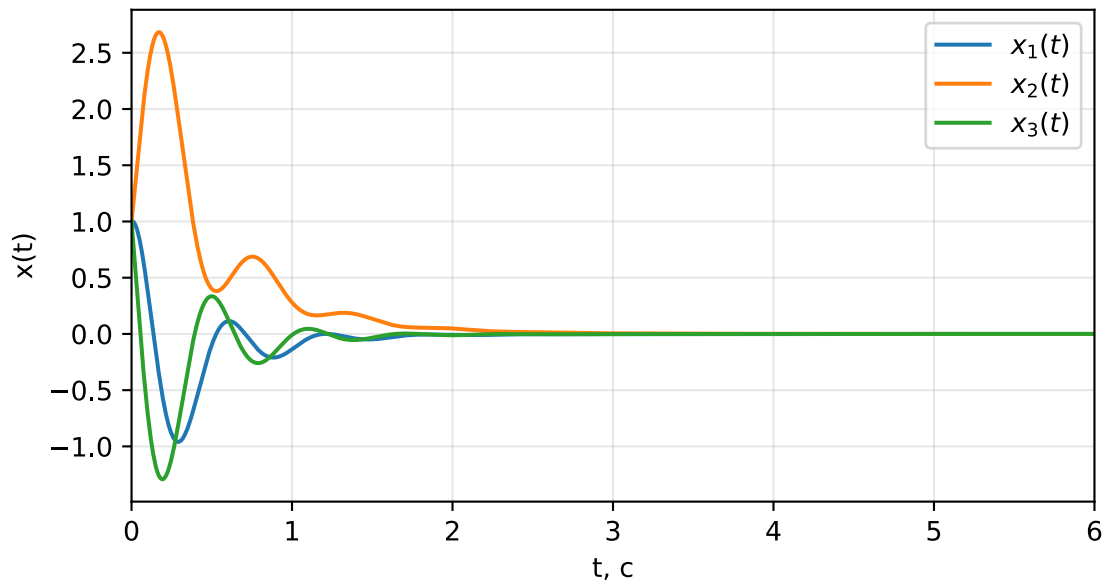


Рисунок 4. Компоненты состояния $x(t)$ для $\alpha_1 = 1$ и регулятора K_1

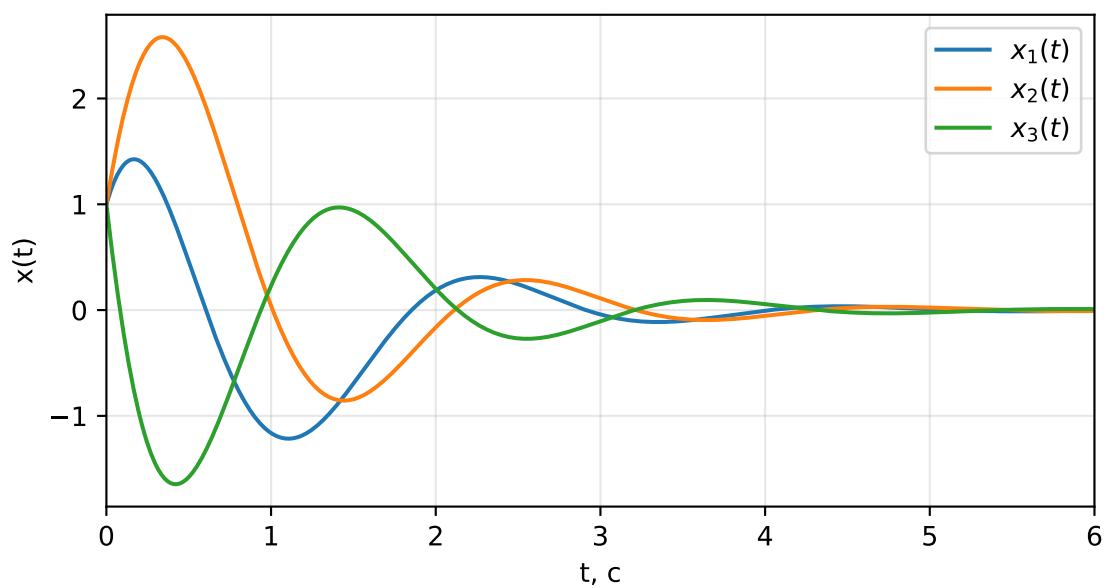


Рисунок 5. Компоненты состояния $x(t)$ для $\alpha_1 = 1$ и регулятора K_2

По графикам видно, что оба регулятора обеспечивают затухание состояния, однако вариант K_2 действует заметно мягче: управление меньше по модулю, а переходный процесс менее «жесткий». Регулятор K_1 создает больший запас устойчивости, но делает систему более быстрой и более энергозатратной.

Для $\alpha_2 = 2$

Получено допустимое решение без дополнительной минимизации:

$$K_1 = [-8.190902 \quad -1.594022 \quad 2.713343].$$

Собственные числа матрицы $A + BK_1$:

$$\sigma(A + BK_1) = \{-2.883536 + 12.650938i, -2.883536 - 12.650938i, -2\}.$$

Для варианта с минимизацией:

$$K_2 = [-2.063850 \quad -1.127764 \quad -2.064026].$$

Собственные числа матрицы $A + BK_2$:

$$\sigma(A + BK_2) = \{-2.000112 + 3.957058i, -2.000112 - 3.957058i, -2\}.$$

В этом случае минимизация вновь прижимает управляемые собственные числа к границе $\Re \lambda = -2$.

Начальные значения управления:

$$u_1(0) = K_1 x(0) = -7.071581, \quad u_2(0) = K_2 x(0) = -5.255640.$$

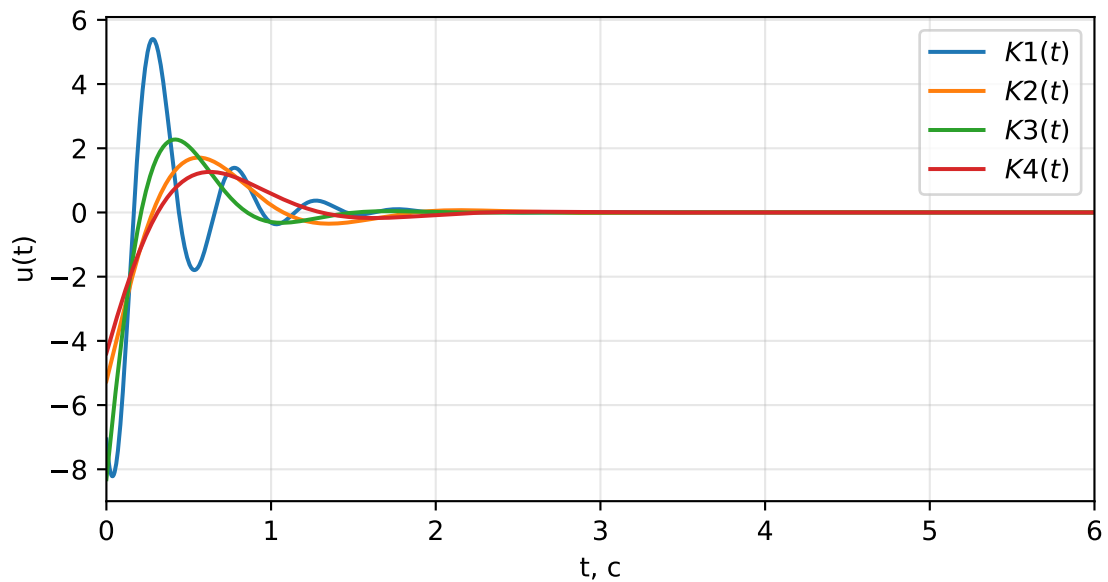


Рисунок 6. Управление $u(t)$ для $\alpha_2 = 2$ и регуляторов K_1, K_2, K_3, K_4

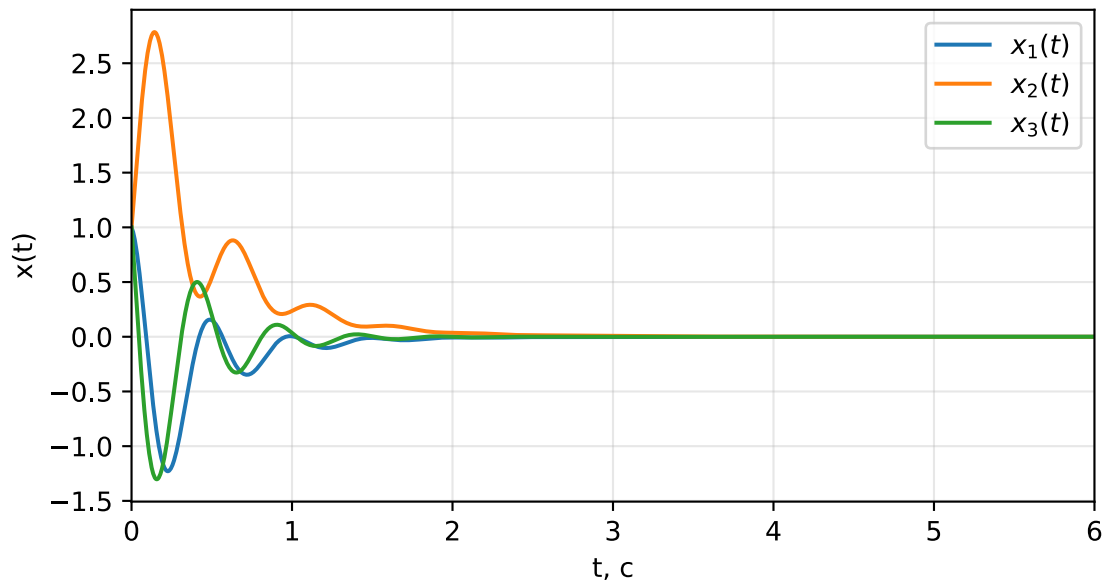


Рисунок 7. Компоненты состояния $x(t)$ для $\alpha_2 = 2$ и регулятора K_1

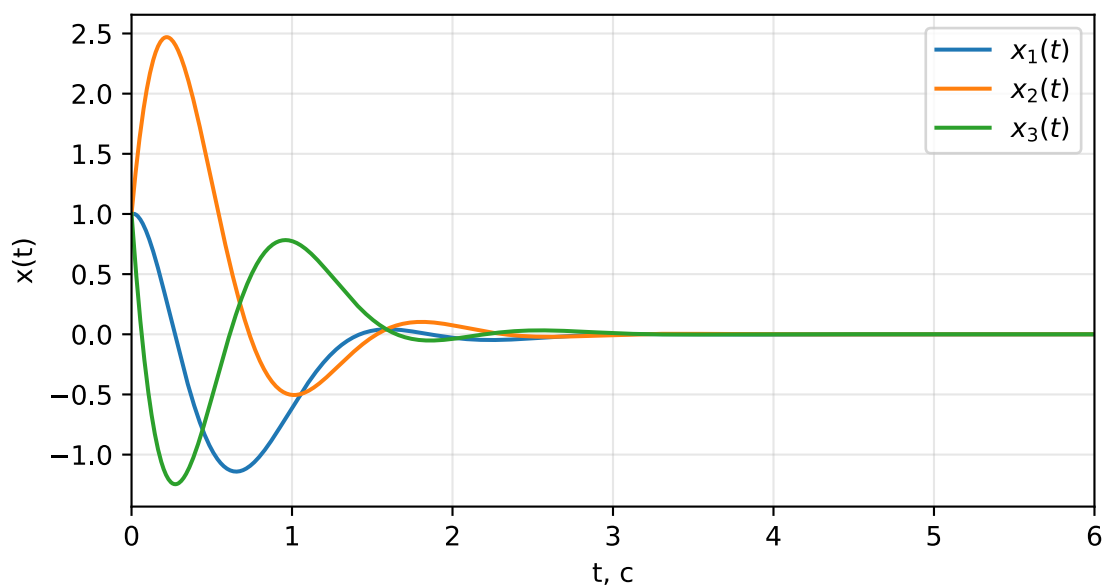


Рисунок 8. Компоненты состояния $x(t)$ для $\alpha_2 = 2$ и регулятора K_2

При $\alpha_2 = 2$ различие между решениями становится еще заметнее. Регулятор K_2 почти прижимает управляемые моды к границе $\Re \lambda = -2$, поэтому управление остается умеренным, тогда как K_1 уводит моды левее и за счет этого дает более резкий отклик.

6) Синтез при помощи матричного уравнения типа Риккати

Используем уравнение

$$A^T P + P A + Q - 2 P B B^T P + 2 \alpha P = 0, \quad K = -B^T P,$$

так как в задании заданы $\nu = 2$ и $R = 1$.

В этом подходе неизвестной является симметричная матрица P , а матрица регулятора вычисляется уже после решения уравнения. В отличие от LMI-постановки, здесь не выбирается произвольное допустимое решение из выпуклого множества, а используется конкретное решение матричного уравнения, зависящее от выбранной матрицы Q .

Для численного решения переходим к стандартному непрерывному CARE для матрицы

$$A_\alpha = A + \alpha I, \quad B_\alpha = \sqrt{2}B.$$

Тогда задача сводится к уравнению

$$A_\alpha^T P + P A_\alpha - P B_\alpha B_\alpha^T P + Q = 0.$$

Такой переход удобен тем, что позволяет использовать стандартный численный решатель CARE, а сама задача остается эквивалентной исходному уравнению для данного α .

Для $\alpha_1 = 1$

При $Q = I$ получаем:

$$K_3 = [-2.399867 \quad -0.755326 \quad -1.808034].$$

$$\sigma(A + BK_3) = \{-2.452575 + 4.117101i, -2.452575 - 4.117101i, -2\}.$$

При $Q = 0$ получаем:

$$K_4 = [-1.076923 \quad -0.153846 \quad -1.076923].$$

$$\sigma(A + BK_4) = \{-1 + 2.236068i, -1 - 2.236068i, -2\}.$$

Вариант $Q = 0$ снова дает размещение на самой границе требуемой степени устойчивости, а при $Q = I$ управляемые моды уходят левее.

Начальные значения управления:

$$u_3(0) = K_3 x(0) = -4.963227, \quad u_4(0) = K_4 x(0) = -2.307692.$$

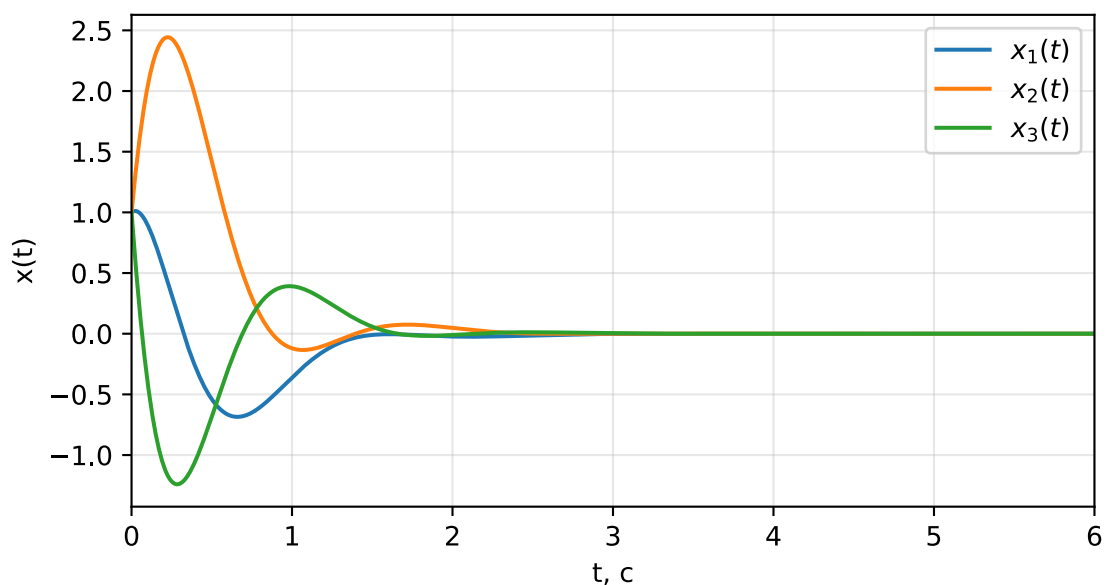


Рисунок 9. Компоненты состояния $x(t)$ для $\alpha_1 = 1$ и регулятора K_3

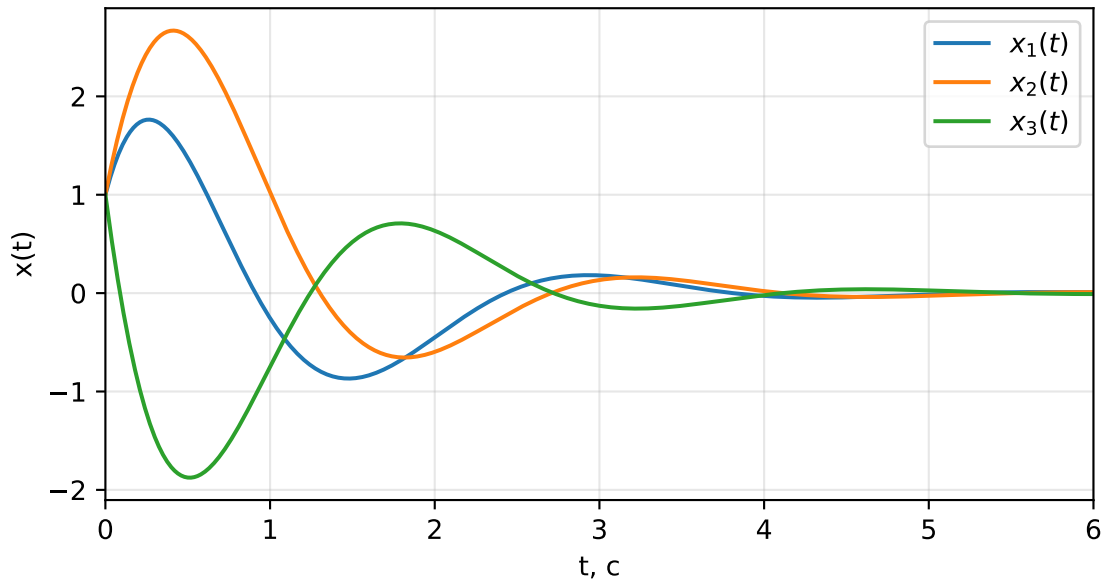


Рисунок 10. Компоненты состояния $x(t)$ для $\alpha_1 = 1$ и регулятора K_4

Для риккатиового синтеза наблюдается та же закономерность, что и для LMI: вариант с $Q = 0$ дает более экономичное по управлению решение, а вариант с $Q = I$ создает дополнительный запас устойчивости ценой более сильного управляющего воздействия.

Для $\alpha_2 = 2$

При $Q = I$ получаем:

$$K_3 = [-3.057657 \quad -2.125000 \quad -3.125000].$$

$$\sigma(A + BK_3) = \{-3.057657 + 4.843339i, -3.057657 - 4.843339i, -2\}.$$

При $Q = 0$ получаем:

$$K_4 = [-1.846154 \quad -0.692308 \quad -1.846154].$$

$$\sigma(A + BK_4) = \{-2 + 3.162278i, -2 - 3.162278i, -2\}.$$

Здесь, как и для α_1 , матрица $Q = 0$ приводит к граничному варианту, а матрица $Q = I$ дает дополнительный запас по вещественной части спектра.

Начальные значения управления:

$$u_3(0) = K_3 x(0) = -8.307657, \quad u_4(0) = K_4 x(0) = -4.384616.$$

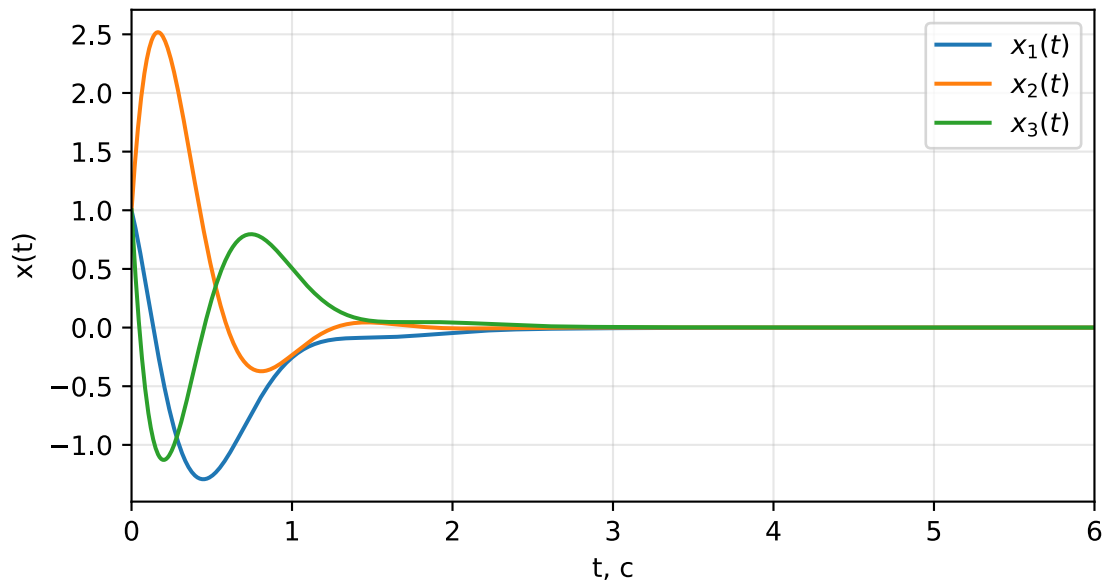


Рисунок 11. Компоненты состояния $x(t)$ для $\alpha_2 = 2$ и регулятора K_3

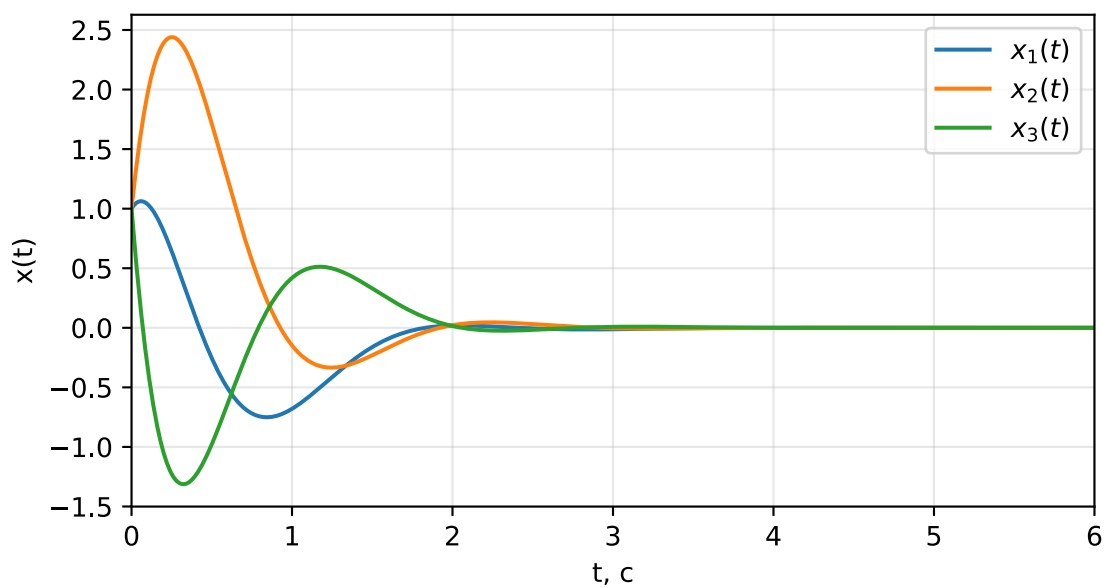


Рисунок 12. Компоненты состояния $x(t)$ для $\alpha_2 = 2$ и регулятора K_4

При максимальной достижимой степени устойчивости $\alpha_2 = 2$ риккатиевые решения также хорошо разделяются по характеру: K_4 остается граничным и более «мягким», а K_3 смещает управляемые моды левее и приводит к более интенсивному управлению.

Регулятор	$\max \Re \lambda$	$u(0)$	$\max u(t) $
K_1	-2.864316	-5.064862	6.145850
K_2	-1.001094	-2.788929	2.788929
K_3	-2.452575	-4.963227	4.963227
K_4	-1.000000	-2.307692	2.307692

Таблица 1. Сравнение регуляторов для $\alpha_1 = 1$

Регулятор	$\max \Re \lambda$	$u(0)$	$\max u(t) $
K_1	-2.883536	-7.071581	8.217195
K_2	-2.000112	-5.255640	5.255640
K_3	-3.057657	-8.307657	8.307657
K_4	-2.000000	-4.384616	4.384616

Таблица 2. Сравнение регуляторов для $\alpha_2 = 2$

7) Сопоставление результатов

По результатам расчетов и моделирования можно сделать следующие выводы по заданию:

- ограничение на достижимую степень устойчивости полностью определяется неуправляемой модой $\lambda = -2$, поэтому выше $\alpha = 2$ подняться нельзя;
- во всех четырех методах для каждого выбранного α требуемая степень устойчивости действительно обеспечивается;
- варианты с дополнительной минимизацией и варианты с $Q = 0$ систематически прижимают управляемые собственные числа к границе допустимой области и уменьшают начальный бросок управления;
- варианты без минимизации и варианты с $Q = I$ дают больший запас устойчивости, но за это приходится платить более крупными коэффициентами регулятора и более высоким управлением.

Итоговый вывод по первому заданию состоит в том, что обе немодальные постановки успешно решают задачу обеспечения заданной степени устойчивости, но приводят к разным по характеру регуляторам. Если важен дополнительный запас по вещественной части спектра, то предпочтительнее решения без минимизации и с $Q = I$. Если же важнее ограничить управляющее воздействие и избежать слишком резкого переходного процесса, то практичнее выглядят решения с минимизацией в LMI-подходе и решения с $Q = 0$ в риккатиевом подходе.

Таким образом, в этом задании подтверждается основная идея немодального синтеза: задается не точный набор полюсов, а требуемая область их расположения, внутри которой можно выбирать регулятор в зависимости от желаемого компромисса между быстродействием и величиной управления.

Задание 2. Управление по выходу с заданной степенью устойчивости

Для варианта 15 в Задании 2 используется набор №15:

$$A = \begin{bmatrix} 6 & 0 & -12 & 6 \\ 0 & 6 & -6 & 12 \\ -12 & -6 & 6 & 0 \\ 6 & 12 & 0 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 6 \\ 12 \\ 6 \\ 4 \end{bmatrix},$$
$$C = \begin{bmatrix} -6 & 6 & 6 & 6 \\ 3 & 0 & 0 & 3 \end{bmatrix}.$$

Рассматривается система

$$\dot{x} = Ax + Bu, \quad y = Cx.$$

Наблюдатель и закон управления:

$$\dot{\hat{x}} = A\hat{x} + Bu + L(C\hat{x} - y), \quad u = K\hat{x}.$$

В данном задании требуется:

- определить свойства системы (управляемость, стабилизируемость, наблюдаемость, обнаруживаемость);
- установить достижимые степени устойчивости регулятора α_K и сходимости наблюдателя α_L ;
- выбрать не менее трех наборов (α_K, α_L) , включая случаи $\alpha_K = \alpha_L$, $\alpha_K > \alpha_L$ и $\alpha_K < \alpha_L$;
- синтезировать матрицы K и L для каждого набора и проверить спектральные требования;
- выполнить моделирование и сравнить влияние соотношения α_K и α_L на поведение системы.

1) Собственные числа, управляемость и наблюдаемость

Собственные числа матрицы A :

$$\sigma(A) = \{24, -12, 12, 0\}.$$

Матрица управляемости:

$$AB = \begin{bmatrix} -12 \\ 84 \\ -108 \\ 204 \end{bmatrix}, \quad A^2B = \begin{bmatrix} 2448 \\ 3600 \\ -1008 \\ 2160 \end{bmatrix}, \quad A^3B = \begin{bmatrix} 39744 \\ 53568 \\ -57024 \\ 70848 \end{bmatrix},$$
$$U = [B, AB, A^2B, A^3B].$$

$$\det(U) \approx -4.586 \cdot 10^{10} \neq 0 \Rightarrow \text{rank}(U) = 4.$$

Следовательно, система полностью управляема.

Матрица наблюдаемости:

$$CA = \begin{bmatrix} -72 & 72 & 72 & 72 \\ 36 & 36 & -36 & 36 \end{bmatrix},$$

$$CA^2 = \begin{bmatrix} -864 & 864 & 864 & 864 \\ 864 & 864 & -864 & 864 \end{bmatrix},$$

$$CA^3 = \begin{bmatrix} -10368 & 10368 & 10368 & 10368 \\ 20736 & 20736 & -20736 & 20736 \end{bmatrix},$$

$$V = \begin{bmatrix} -6 & 6 & 6 & 6 \\ 3 & 0 & 0 & 3 \\ -72 & 72 & 72 & 72 \\ 36 & 36 & -36 & 36 \\ -864 & 864 & 864 & 864 \\ 864 & 864 & -864 & 864 \\ -10368 & 10368 & 10368 & 10368 \\ 20736 & 20736 & -20736 & 20736 \end{bmatrix}, \quad \text{rank}(V) = 3 < 4.$$

Следовательно, система не полностью наблюдаема.

По Хаугусу:

$$\text{rank}(H_o(24)) = \text{rank}(H_o(12)) = \text{rank}(H_o(0)) = 4, \quad \text{rank}(H_o(-12)) = 3.$$

Итак, система полностью управляема и стабилизируема, не полностью наблюдаема, но обнаруживаема (ненаблюдаемая мода $\lambda = -12$ устойчива).

2) Достижимые степени устойчивости и сходимости

Для регулятора $u = Kx$ система полностью управляема, поэтому можно обеспечить любую желаемую степень устойчивости

$$\alpha_K > 0.$$

Для наблюдателя полной размерности ненаблюдаемая мода -12 не может быть смещена матрицей L . Поэтому достижимая степень сходимости ограничена:

$$0 < \alpha_L \leq 12, \quad \alpha_{L, \max} = 12.$$

Выберем значения

$$\alpha \in \{8, 12\}.$$

Они допустимы как для регулятора, так и для наблюдателя, при этом 12 является максимально достижимым для наблюдателя.

3) Наборы (α_K, α_L)

Составим три требуемых набора:

1. $\alpha_K = \alpha_L: (8, 8);$
2. $\alpha_K > \alpha_L: (12, 8);$
3. $\alpha_K < \alpha_L: (8, 12).$

4) Синтез K и L по LMI

Для регулятора используем LMI

$$AP + PA^T + 2\alpha_K P + BY_K + Y_K^T B^T \leq 0, \quad K = Y_K P^{-1}, \quad P > 0.$$

Для наблюдателя используем дуальную постановку на системе (A^T, C^T) :

$$A_d = A^T, \quad B_d = C^T,$$

$$A_d P + P A_d^T + 2\alpha_L P + B_d Y_L + Y_L^T B_d^T \leq 0, \quad K_d = Y_L P^{-1}, \quad L = K_d^T, \quad P > 0.$$

Полученные матрицы:

Для $\alpha_K = 8$:

$$K_8 = [-77.702530 \quad 25.013197 \quad 30.607353 \quad -23.254228],$$

$$\sigma(A + BK_8) \approx \{-14.900818 \pm 45.276535i, -10.813991 \pm 5.302478i\}.$$

Для $\alpha_K = 12$:

$$K_{12} = [-239.048896 \quad 121.812456 \quad 36.494413 \quad -87.052328],$$

$$\sigma(A + BK_{12}) \approx \{-22.354063 \pm 60.784750i, -16.539305 \pm 8.089213i\}.$$

Для $\alpha_L = 8$:

$$L_8 = \begin{bmatrix} 0.833333 & -6.666667 \\ -0.833333 & -12.590796 \\ -0.833333 & 12.590796 \\ -0.833333 & -6.666667 \end{bmatrix},$$

$$\sigma(A + L_8 C) = \{-8 \pm 19.040413i, -12, -8\}.$$

Для $\alpha_L = 12$:

$$L_{12} = \begin{bmatrix} 1 & -8 \\ -1 & -17.636644 \\ -1 & 17.636644 \\ -1 & -8 \end{bmatrix},$$

$$\sigma(A + L_{12} C) = \{-12 \pm 23.448632i, -12, -12\}.$$

5) Проверка для выбранных наборов

1. Набор $(\alpha_K, \alpha_L) = (8, 8)$:

$$K = K_8, \quad L = L_8,$$

$$\max \Re \sigma(A + BK) = -10.813991 \leq -8,$$

$$\max \Re \sigma(A + LC) = -8 \leq -8.$$

2. Набор $(\alpha_K, \alpha_L) = (12, 8)$:

$$K = K_{12}, \quad L = L_8,$$

$$\max \Re \sigma(A + BK) = -16.539305 \leq -12,$$

$$\max \Re \sigma(A + LC) = -8 \leq -8.$$

3. Набор $(\alpha_K, \alpha_L) = (8, 12)$:

$$K = K_8, \quad L = L_{12},$$

$$\max \Re \sigma(A + BK) = -10.813991 \leq -8,$$

$$\max \Re \sigma(A + LC) = -12 \leq -12.$$

6) Моделирование

Моделирование выполнено при

$$x(0) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}^T, \quad \hat{x}(0) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}^T.$$

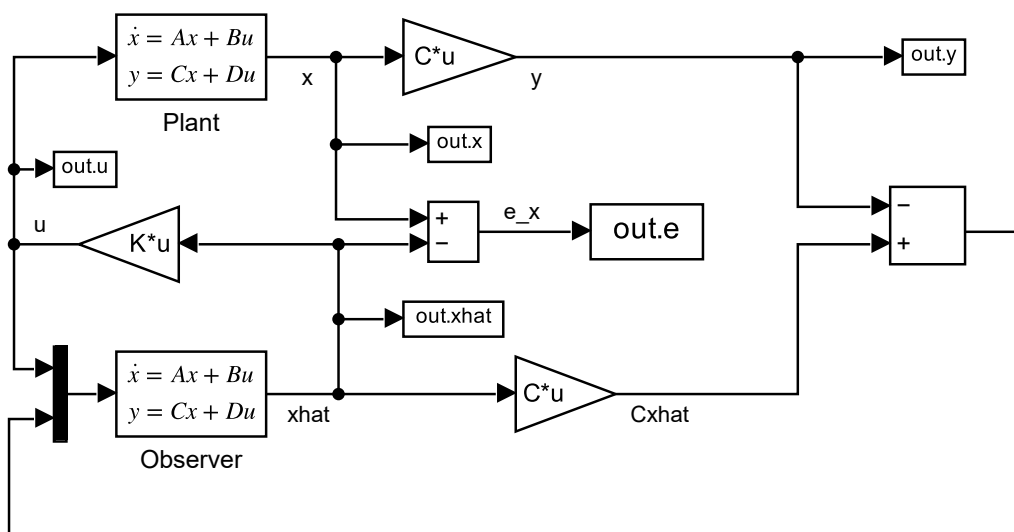


Рисунок 13. Структурная схема моделирования для задания 2

Parameters

A:

A
4x4 double

B:

B
6;12;6;4

C:

eye(4)
4x4 double

D:

zeros(4,1)
0;0;0;0

Initial conditions:

[1;1;1;1]

Рисунок 14. Параметры блока State-Space объекта для задания 2

Parameters

A:

A <4x4 double> ⋮

B:

[B, L] <4x3 double> ⋮

C:

eye(4) <4x4 double> ⋮

D:

zeros(4,3) <4x3 double> ⋮

Initial conditions:

[0;0;0;0] ⋮

Рисунок 15. Параметры блока State-Space наблюдателя для задания 2

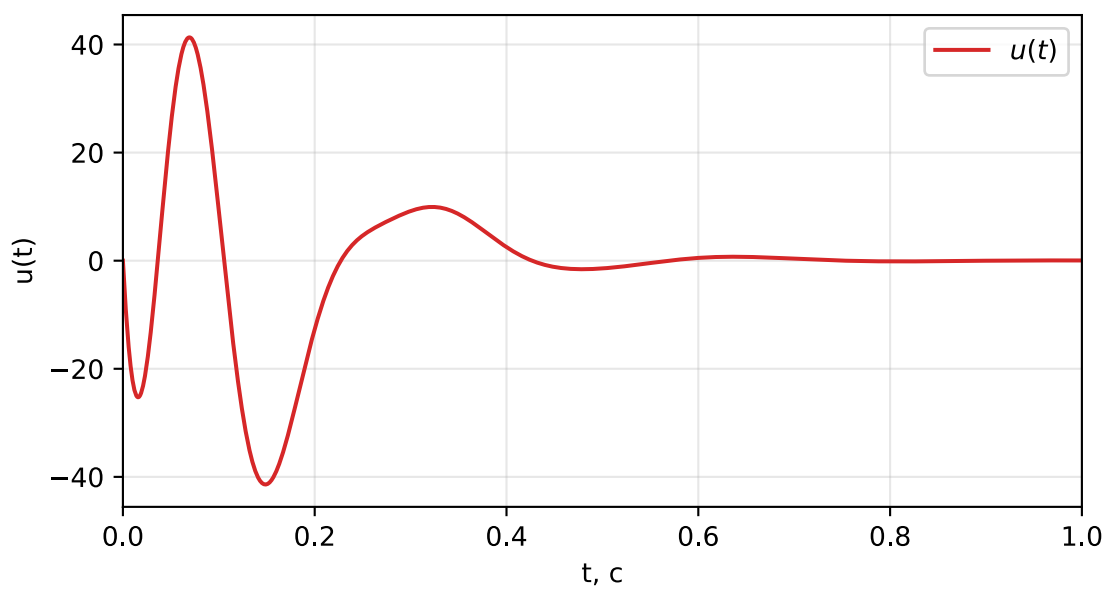


Рисунок 16. Управление $u(t)$ для набора $(\alpha_K, \alpha_L) = (8, 8)$

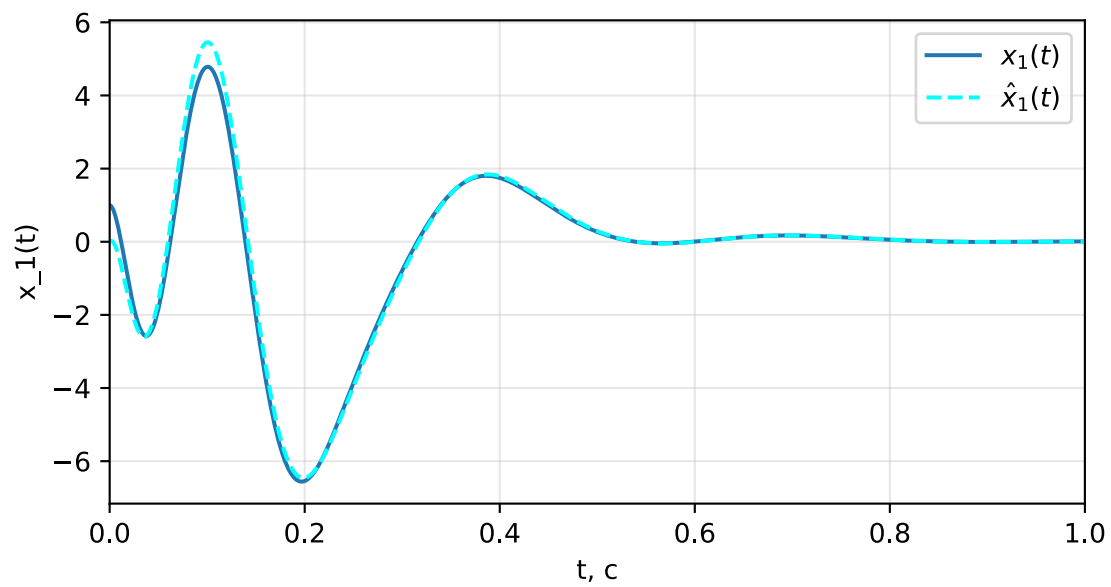


Рисунок 17. Сравнение $x_1(t)$ и $\hat{x}_1(t)$ для набора (8, 8)

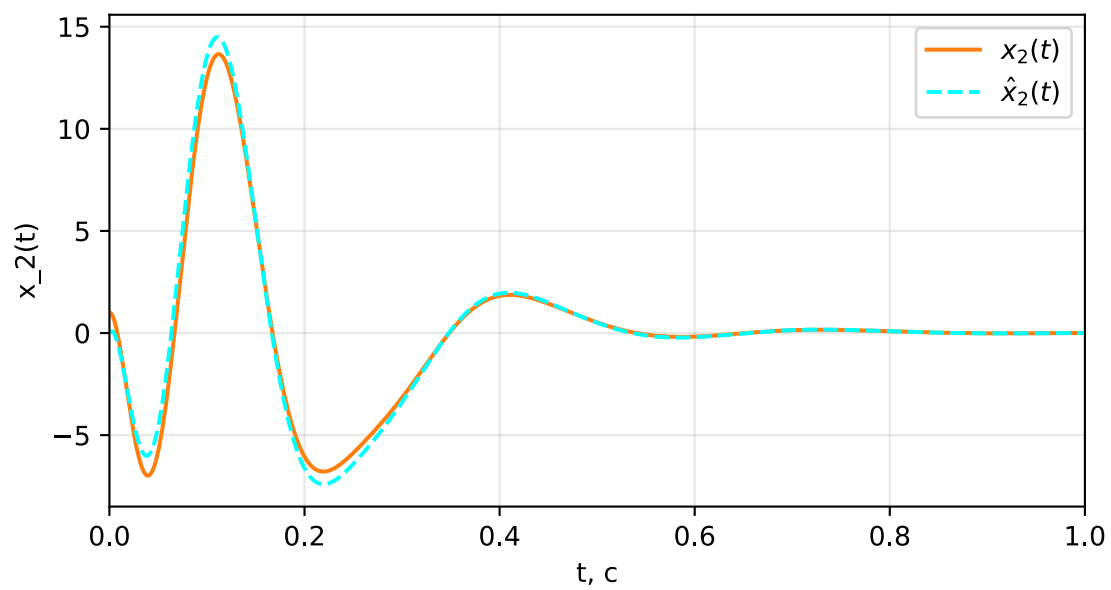


Рисунок 18. Сравнение $x_2(t)$ и $\hat{x}_2(t)$ для набора (8, 8)

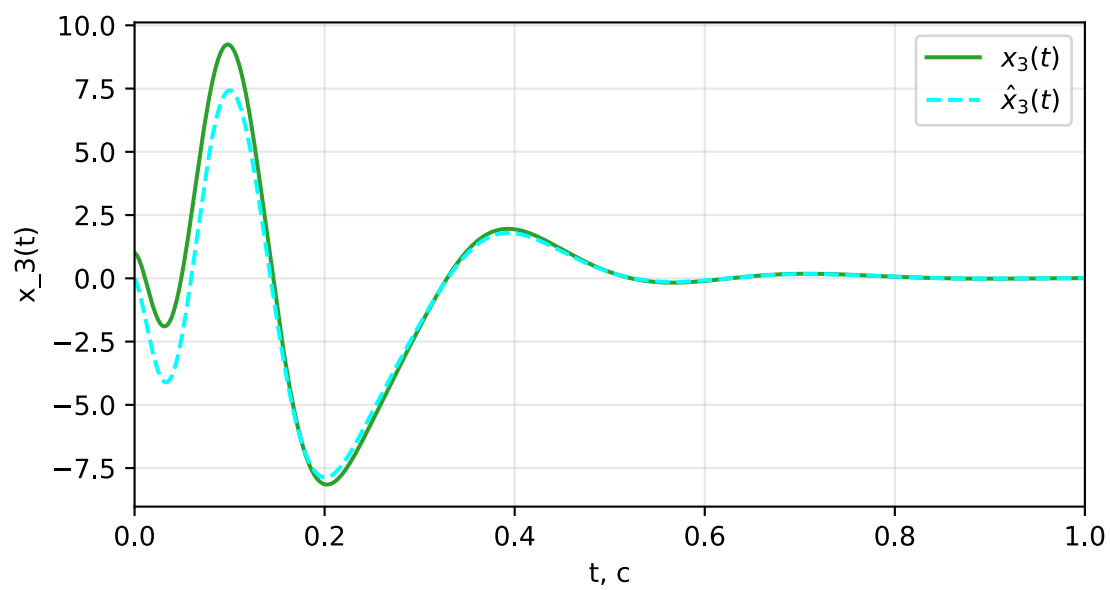


Рисунок 19. Сравнение $x_3(t)$ и $\hat{x}_3(t)$ для набора (8, 8)

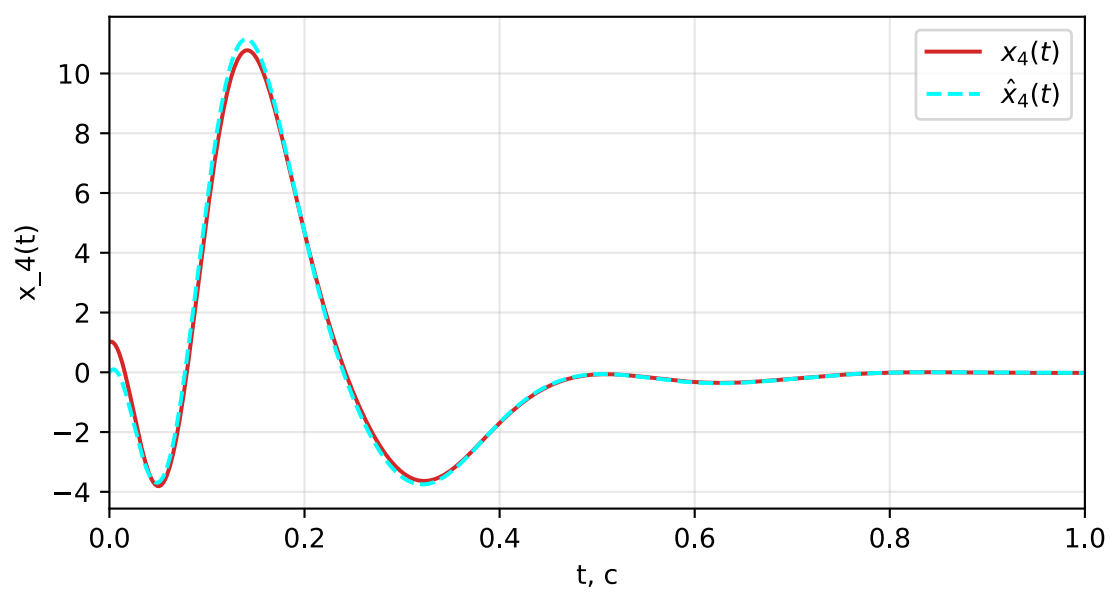


Рисунок 20. Сравнение $x_4(t)$ и $\hat{x}_4(t)$ для набора (8, 8)

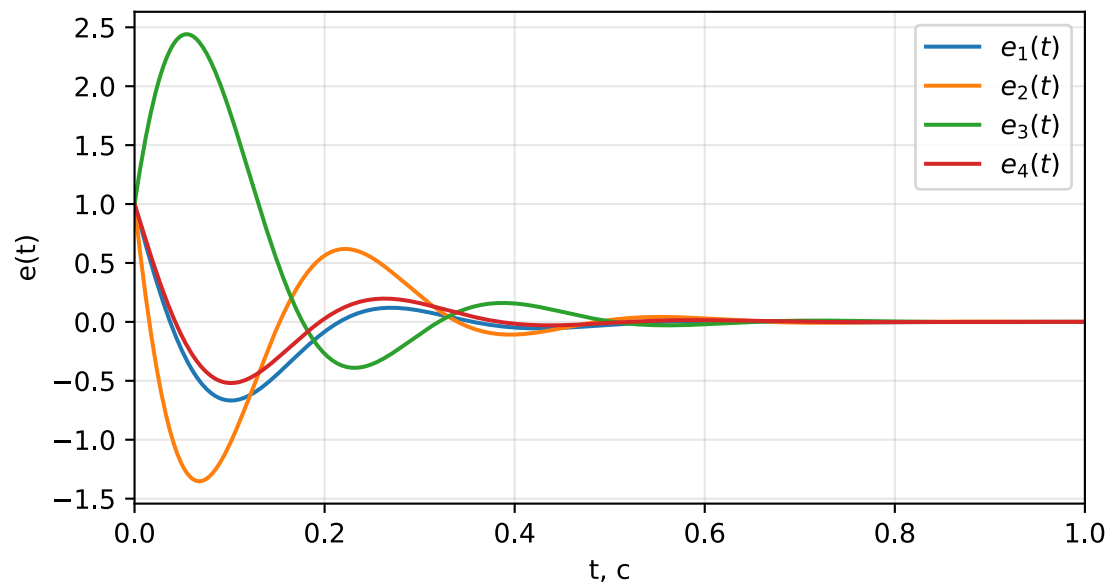


Рисунок 21. Компоненты ошибки наблюдения $e(t)$ для набора $(8, 8)$

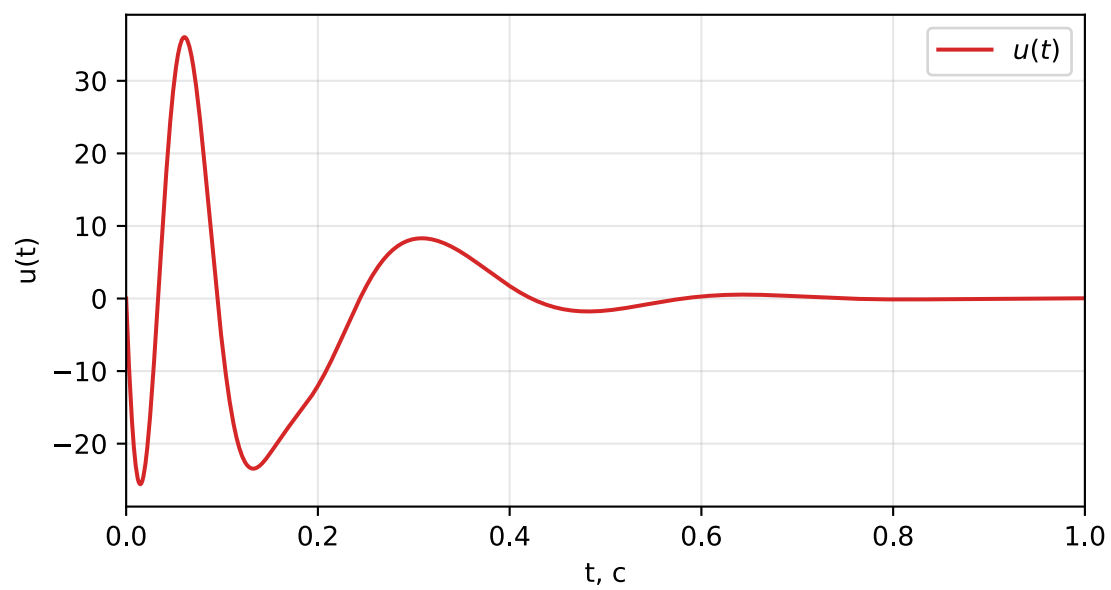


Рисунок 22. Управление $u(t)$ для набора $(\alpha_K, \alpha_L) = (12, 8)$

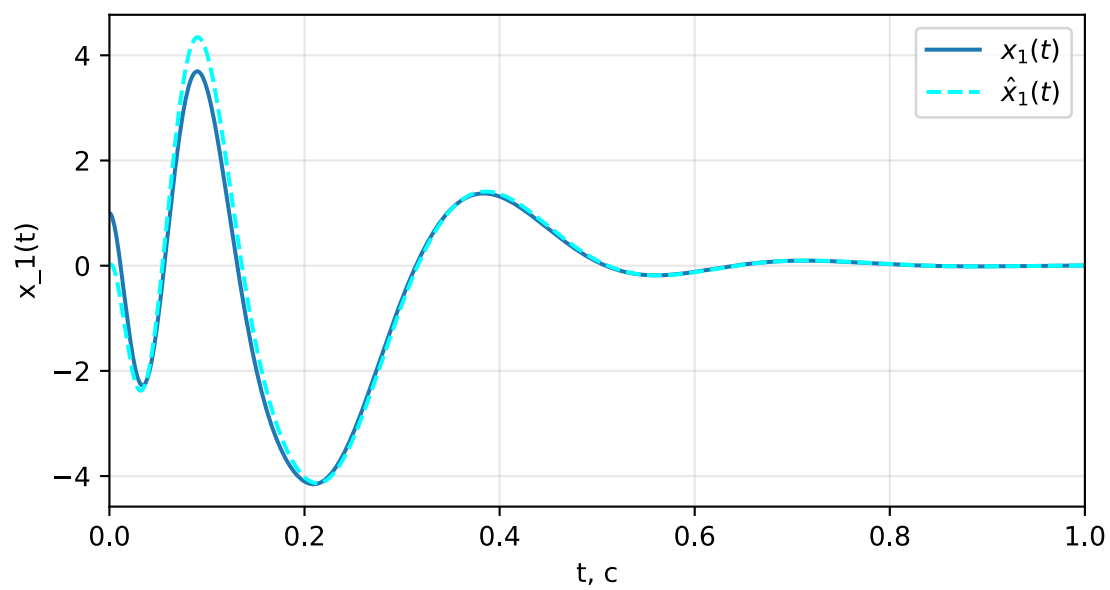


Рисунок 23. Сравнение $x_1(t)$ и $\hat{x}_1(t)$ для набора (12, 8)

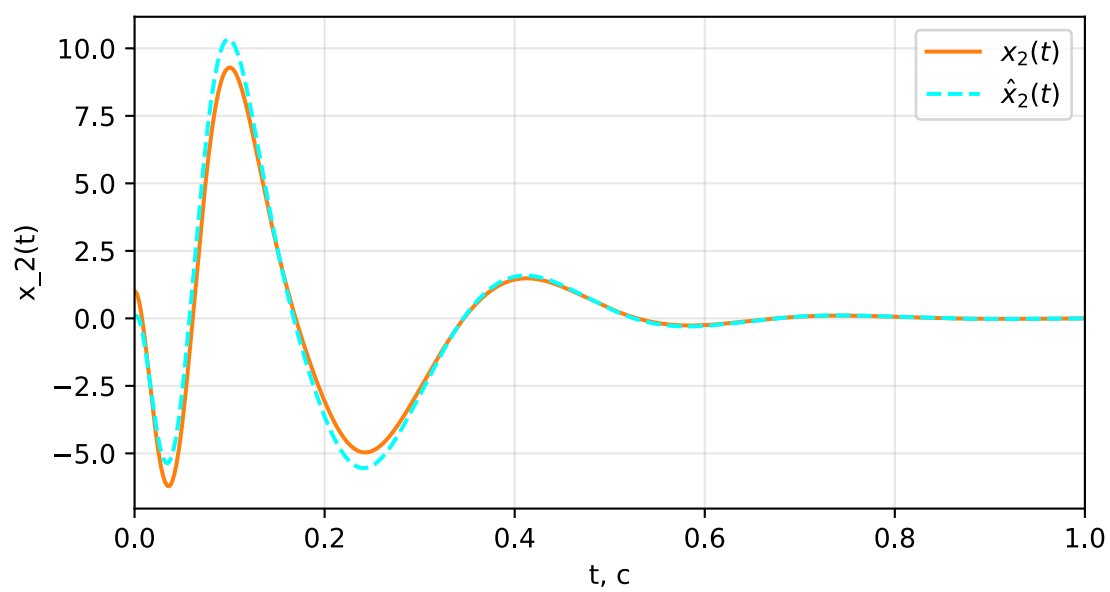


Рисунок 24. Сравнение $x_2(t)$ и $\hat{x}_2(t)$ для набора (12, 8)

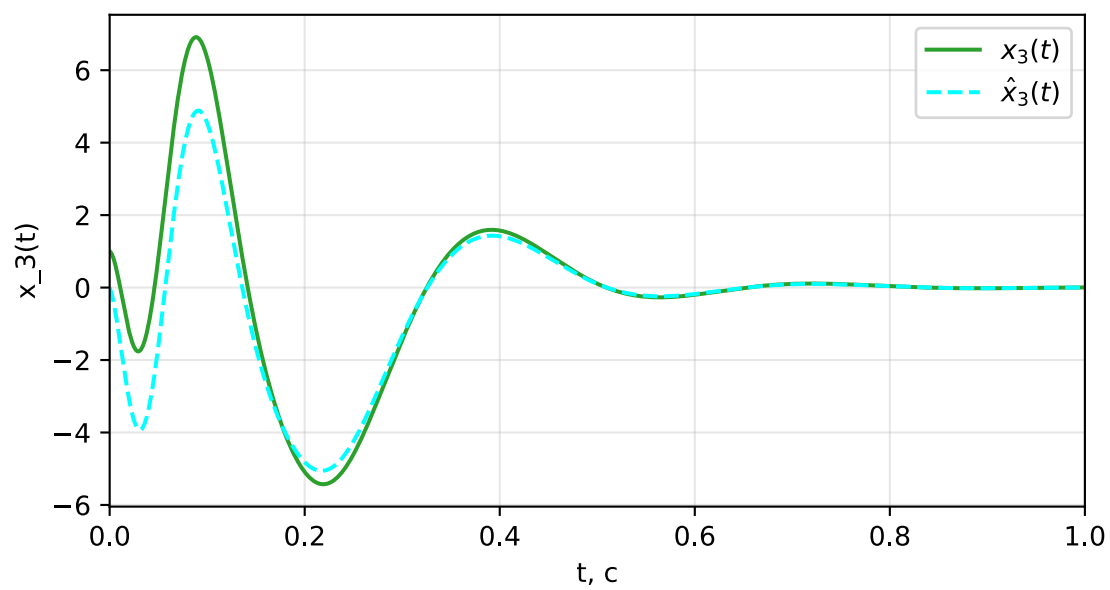


Рисунок 25. Сравнение $x_3(t)$ и $\hat{x}_3(t)$ для набора (12, 8)

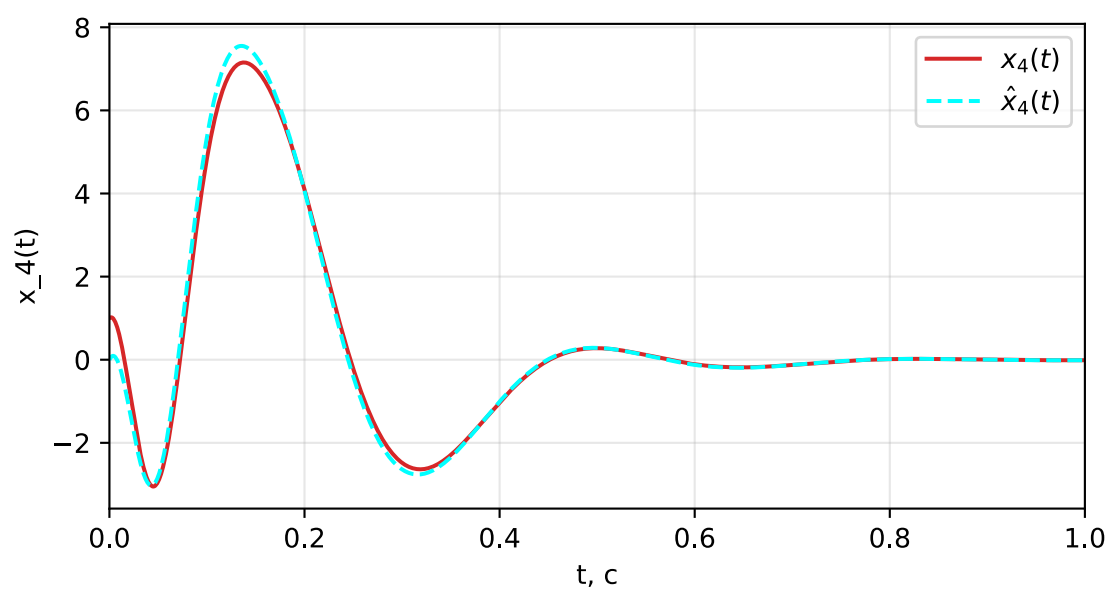


Рисунок 26. Сравнение $x_4(t)$ и $\hat{x}_4(t)$ для набора (12, 8)

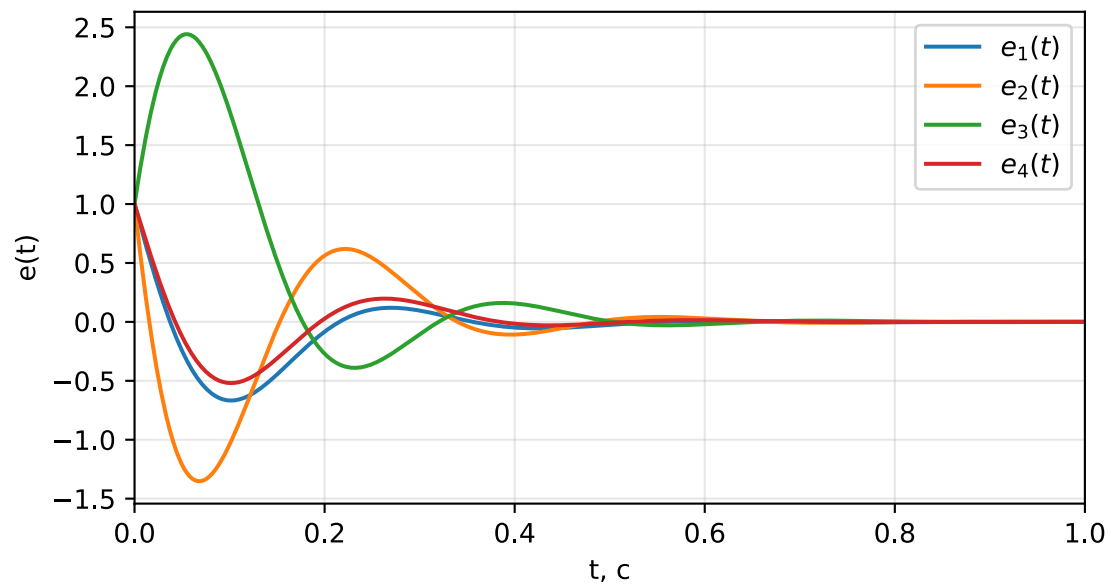


Рисунок 27. Компоненты ошибки наблюдения $e(t)$ для набора $(12, 8)$

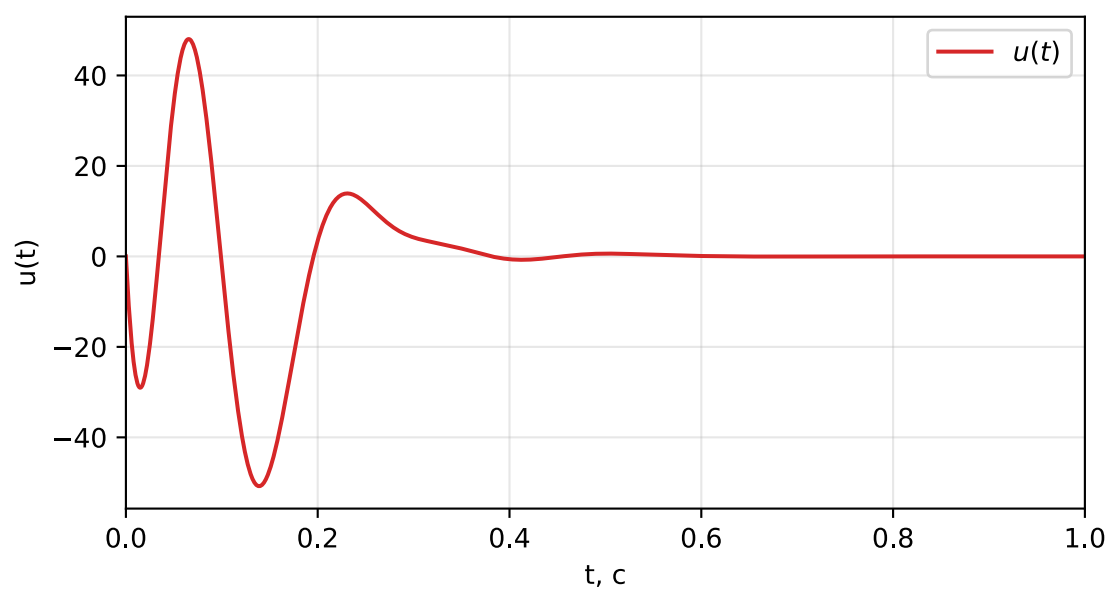


Рисунок 28. Управление $u(t)$ для набора $(\alpha_K, \alpha_L) = (8, 12)$

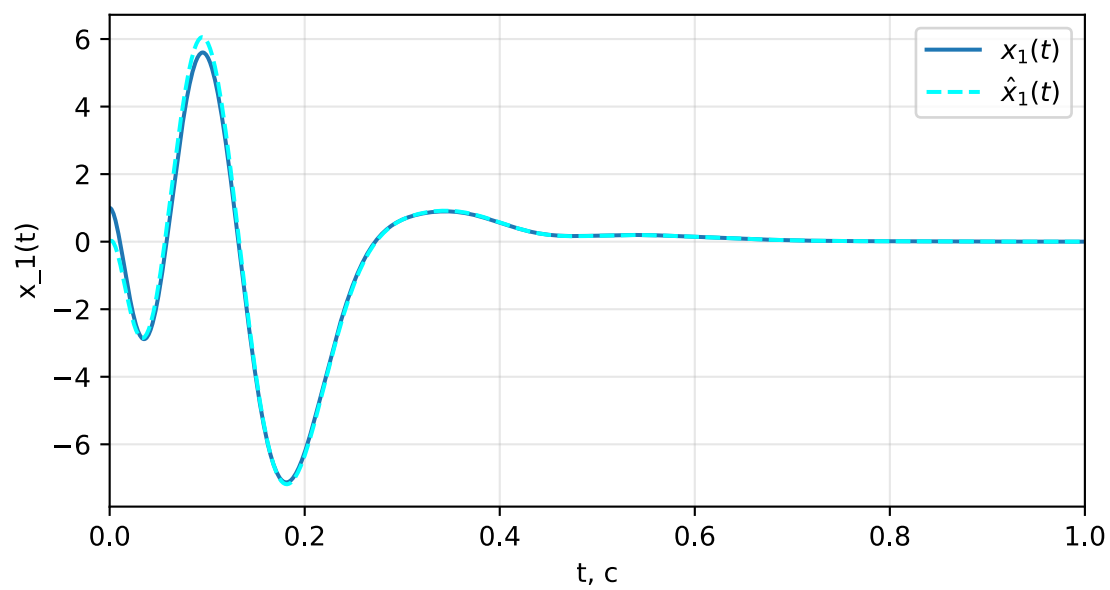


Рисунок 29. Сравнение $x_1(t)$ и $\hat{x}_1(t)$ для набора (8, 12)

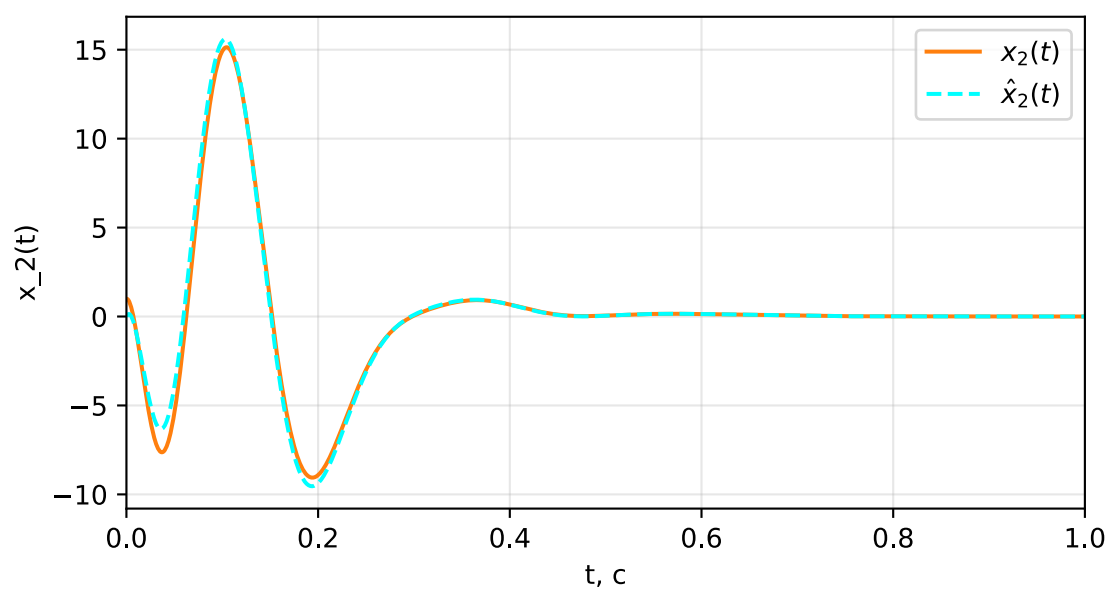


Рисунок 30. Сравнение $x_2(t)$ и $\hat{x}_2(t)$ для набора (8, 12)

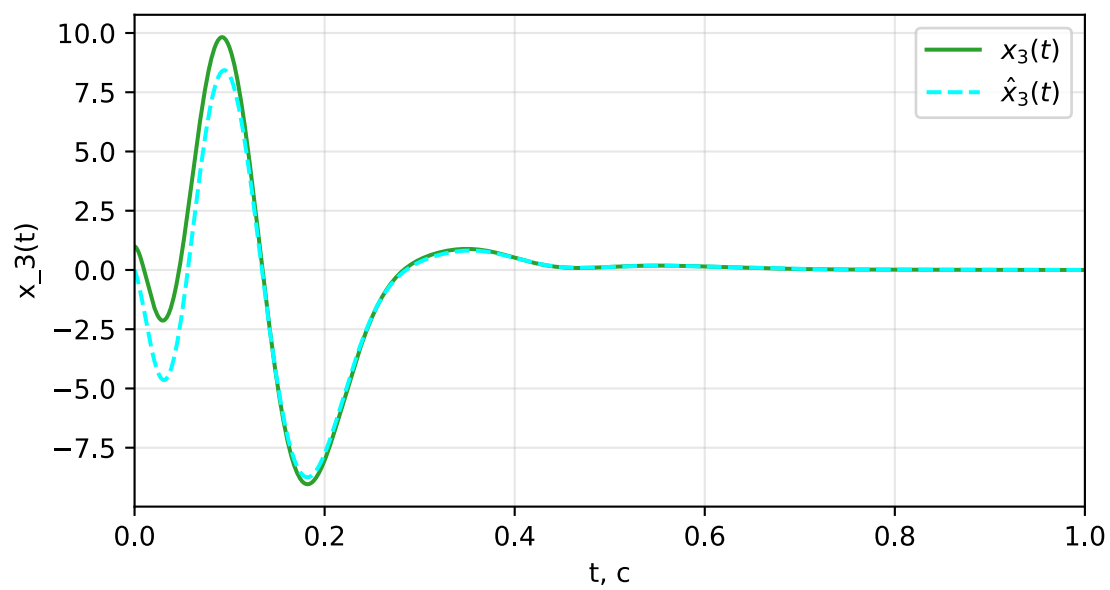


Рисунок 31. Сравнение $x_3(t)$ и $\hat{x}_3(t)$ для набора (8, 12)

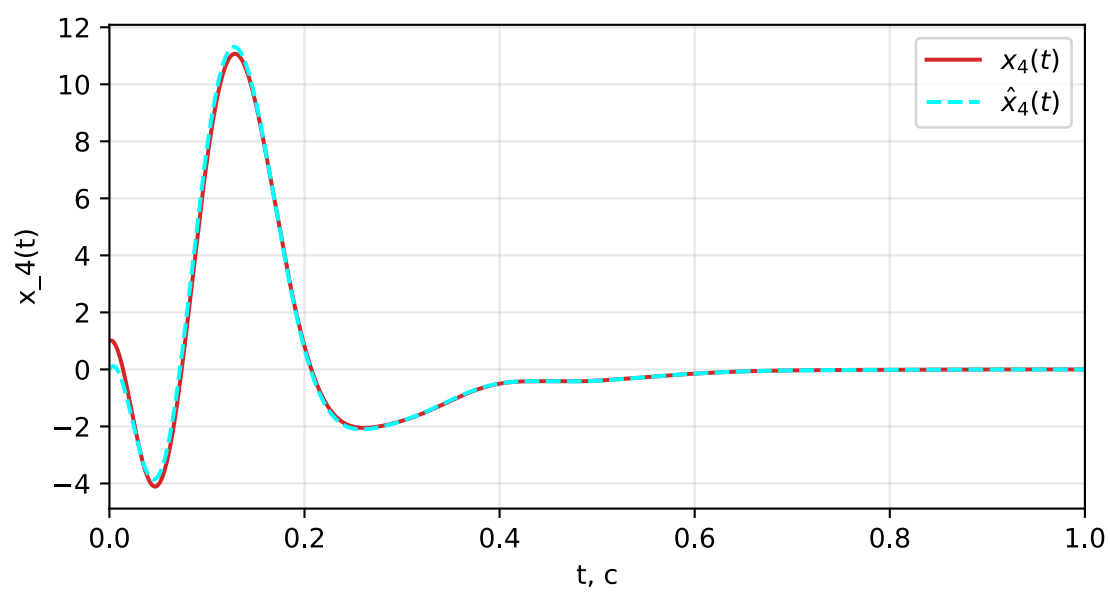


Рисунок 32. Сравнение $x_4(t)$ и $\hat{x}_4(t)$ для набора (8, 12)

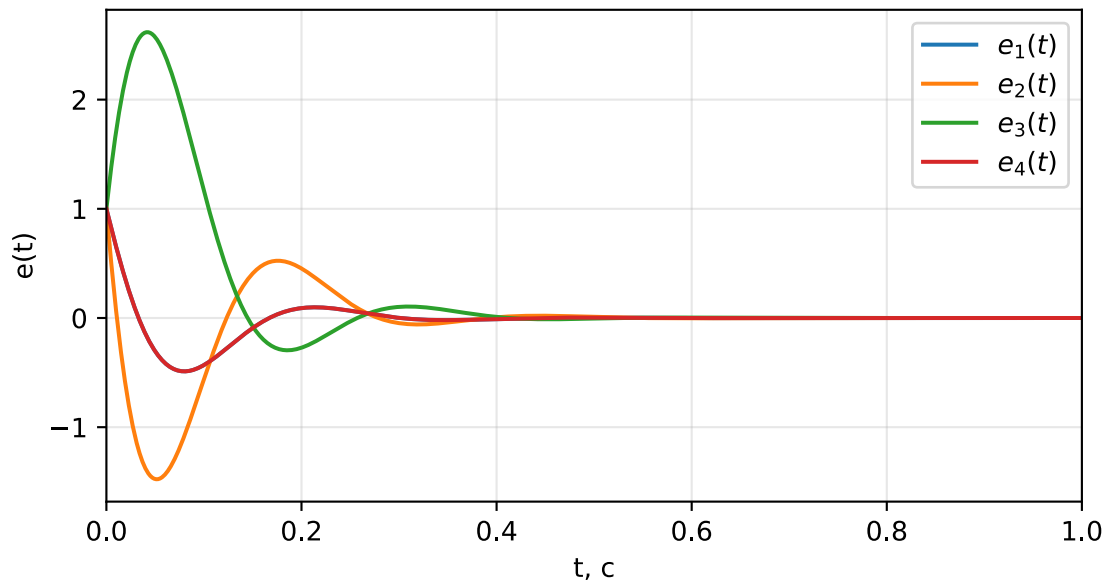


Рисунок 33. Компоненты ошибки наблюдения $e(t)$ для набора (8, 12)

7) Сопоставление наборов (α_K, α_L)

По результатам расчетов и моделирования:

- набор (8, 8) дает сбалансированную динамику регулятора и наблюдателя;
- набор (12, 8) делает регулятор более «жестким», управление возрастает, а скорость сходимости наблюдателя остается прежней;
- набор (8, 12) ускоряет наблюдатель относительно регулятора, ошибка наблюдения затухает быстрее, но возрастает чувствительность и амплитуда переходных участков.

Итог: при управлении по выходу влияние α_K и α_L различается по роли. Увеличение α_K в первую очередь усиливает управляющее воздействие и скорость замкнутой системы, а увеличение α_L — скорость восстановления состояния. Практически целесообразно выбирать их согласованно, избегая слишком большого разрыва между регулятором и наблюдателем.

Набор	α_K	α_L	$\max \Re \sigma(A + BK)$	$\max \Re \sigma(A + LC)$	$\max u(t) $
(8, 8)	8	8	-10.813991	-8	41.411805
(12, 8)	12	8	-16.539305	-8	36.023230
(8, 12)	8	12	-10.813991	-12	50.802254

Таблица 3. Сводное сравнение наборов (α_K, α_L) для задания 2

Задание 3. Регулятор с качественной экспоненциальной устойчивостью

В Задании 3 используется та же система, что и в Задании 1:

$$\dot{x} = Ax + Bu, \quad u = Kx,$$

$$A = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 0 & 2 \\ -2 & -2 & -4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 \\ -2 \\ 2 \end{bmatrix}.$$

Из Задания 1: система не полностью управляема, неуправляемая мода равна $\lambda = -2$ и устойчива.

В данном задании требуется:

- выбрать параметры области качественной экспоненциальной устойчивости (β, r) ;
- синтезировать регулятор для всех четырех наборов (Q, R) из условия;
- проверить расположение спектра $A + BK$ относительно круга с центром $(\beta, 0)$ и радиусом r ;
- выполнить моделирование и сопоставить полученные регуляторы по характеру переходных процессов.

1) Выбор параметров β и r

Выберем параметры качественной экспоненциальной устойчивости:

$$\beta = -2.5, \quad r = 1.$$

Проверка условий:

$$\beta < 0, \quad r > 0, \quad \beta + r = -1.5 < 0.$$

Неуправляемая мода должна лежать внутри круга:

$$|-2 - \beta| = |-2 + 2.5| = 0.5 \leq r.$$

2) Синтез для наборов (Q, R)

Используется уравнение типа Риккати

$$(A + BK - \beta I)^T P (A + BK - \beta I) - r^2 P = -Q,$$

$$K = -(R + B^T P B)^{-1} B^T P (A - \beta I).$$

Рассмотрены четыре набора:

1. $Q = I, R = 1$;
2. $Q = I, R = 0$;
3. $Q = 0, R = 1$;
4. $Q = 0, R = 0$.

Результаты синтеза:

1. $Q = I, R = 1$

$$K_1 = [-1.751669 \quad -0.035967 \quad -1.741567],$$

$$\sigma(A + BK_1) \approx \{-2.457269 \pm 0.134136i, -2\},$$

$$\max|\lambda - \beta| = 0.5 \leq 1.$$

2. $Q = I, R = 0$

$$K_2 = [-1.754348 \quad -0.036043 \quad -1.744228],$$

$$\sigma(A + BK_2) = \{-2.5, -2.425066, -2\},$$

$$\max|\lambda - \beta| = 0.5 \leq 1.$$

3. $Q = 0, R = 1$

$$K_3 = [-1.496120 \quad 0.016532 \quad -1.496120],$$

$$\sigma(A + BK_3) \approx \{-2.008772 \pm 0.871031i, -2\},$$

$$\max|\lambda - \beta| = 1 \leq 1.$$

4. $Q = 0, R = 0$

Для этого набора прямой численный поиск на полной системе не сходится к устойчивому решению, поэтому синтез выполняем на усеченной управляемой подсистеме второго порядка, а затем расширяем регулятор обратно на исходную систему. При этом управляемые собственные числа размещаются на границе круга в точках $\beta - r = -3.5$ и $\beta + r = -1.5$, а неуправляемая мода -2 сохраняется внутри круга.

$$K_4 = [-1.740385 \quad 0.019231 \quad -1.740385], \quad \sigma(A + BK_4) = \{-3.5, -1.5, -2\},$$

$$\max|\lambda - \beta| = 1 \leq 1.$$

3) Проверка на комплексной плоскости

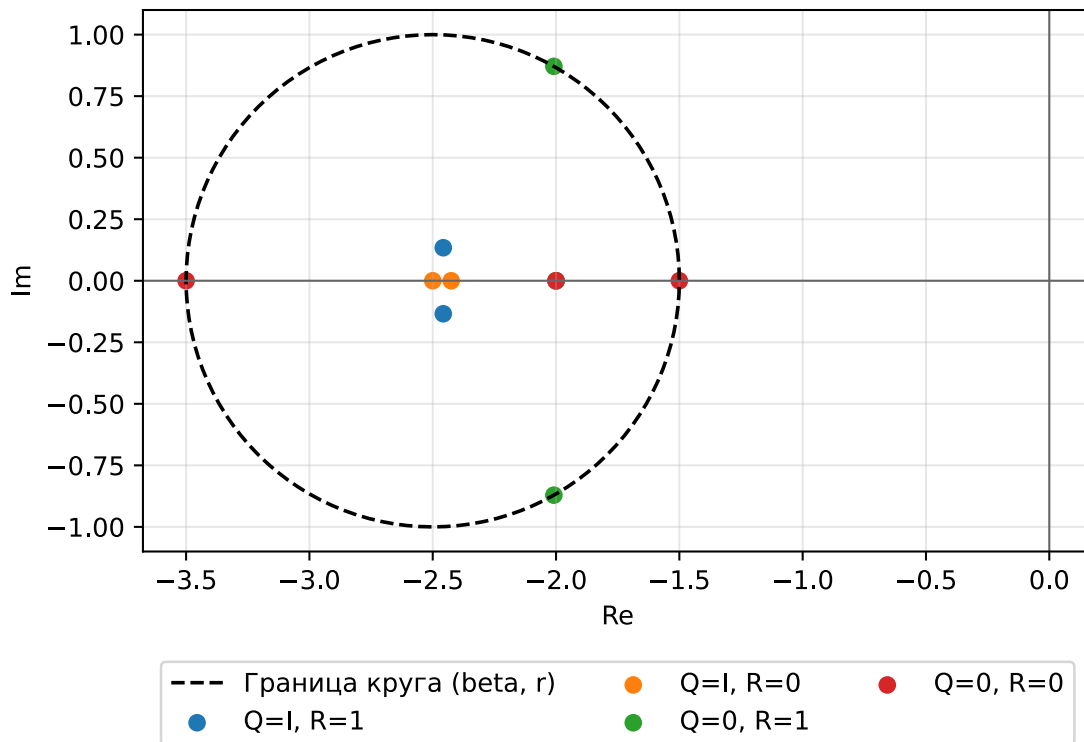


Рисунок 34. Собственные числа $A + BK$ и круг с центром $(\beta, 0)$ радиуса r

По графику видно:

- для наборов $(Q, R) = (I, 1)$ и $(I, 0)$ собственные числа лежат строго внутри требуемого круга;
- для наборов $(Q, R) = (0, 1)$ и $(0, 0)$ управляемые собственные числа лежат на границе круга, а неуправляемое число -2 остается внутри него.

Набор (Q, R)	$\max \lambda - \beta $	В круге	$\max u(t) $	Комментарий
$(I, 1)$	0.5	Да	3.529204	устойчивый, с запасом
$(I, 0)$	0.5	Да	3.534620	устойчивый, близок к $(I, 1)$
$(0, 1)$	1.0	Да (граница)	2.975709	граничный случай
$(0, 0)$	1.0	Да (граница)	3.461538	решение найдено на управляемой части

Таблица 4. Сводное сравнение наборов (Q, R) для задания 3

4) Моделирование

Моделирование выполнено при

$$x(0) = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}^T.$$

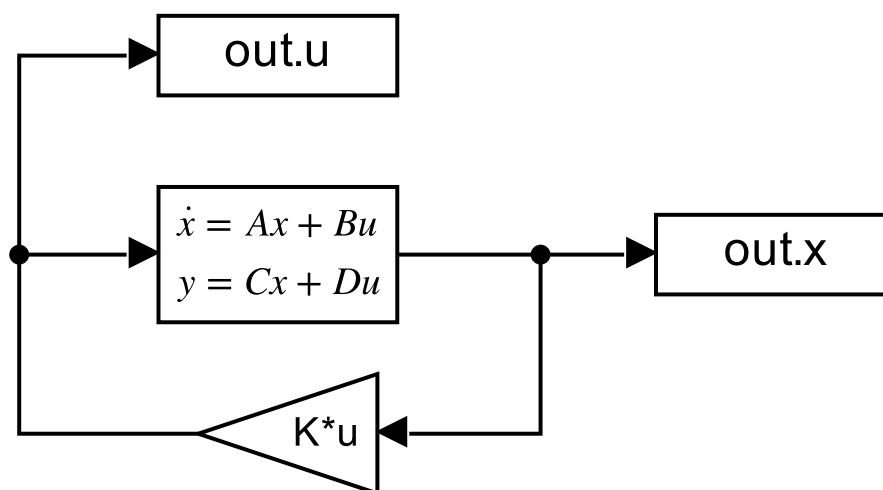


Рисунок 35. Структурная схема моделирования для задания 3

Parameters	
A:	
A	<3x3 double>
B:	
B	[2;-2;2]
C:	
eye(3)	<3x3 double>
D:	
zeros(3,1)	[0;0;0]
Initial conditions:	
[1;1;1]	

Рисунок 36. Параметры блоков модели для задания 3

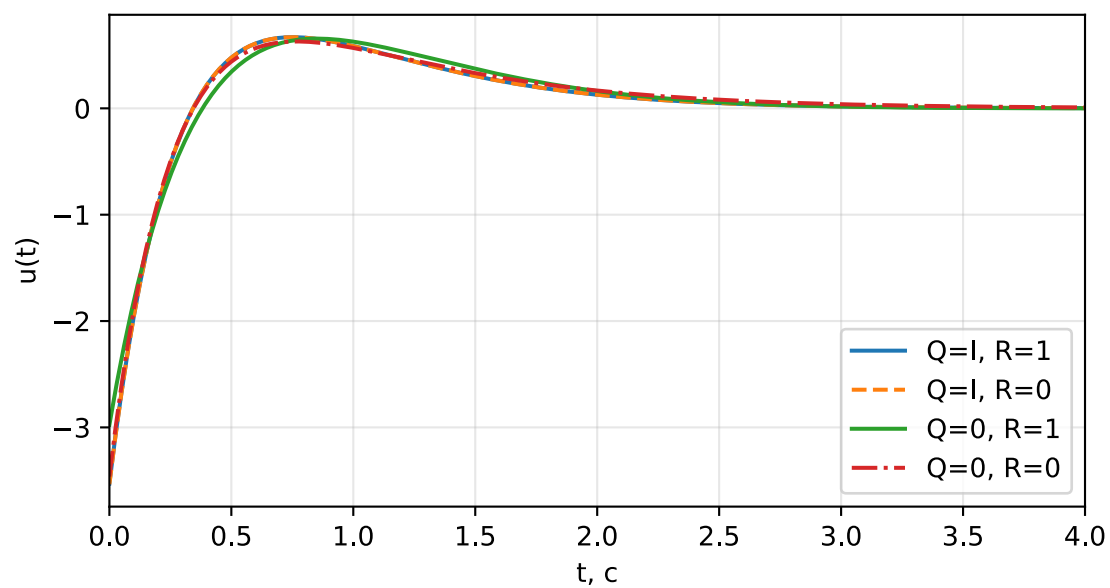


Рисунок 37. Управление $u(t)$ для четырех наборов (Q, R)

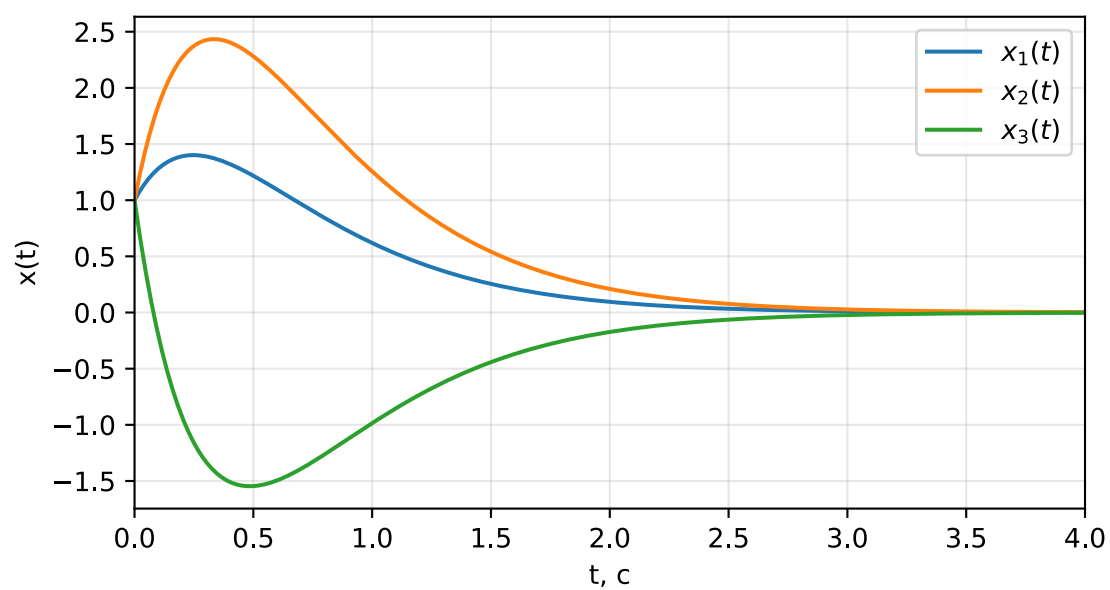


Рисунок 38. Компоненты состояния $x(t)$ для $(Q, R) = (I, 1)$

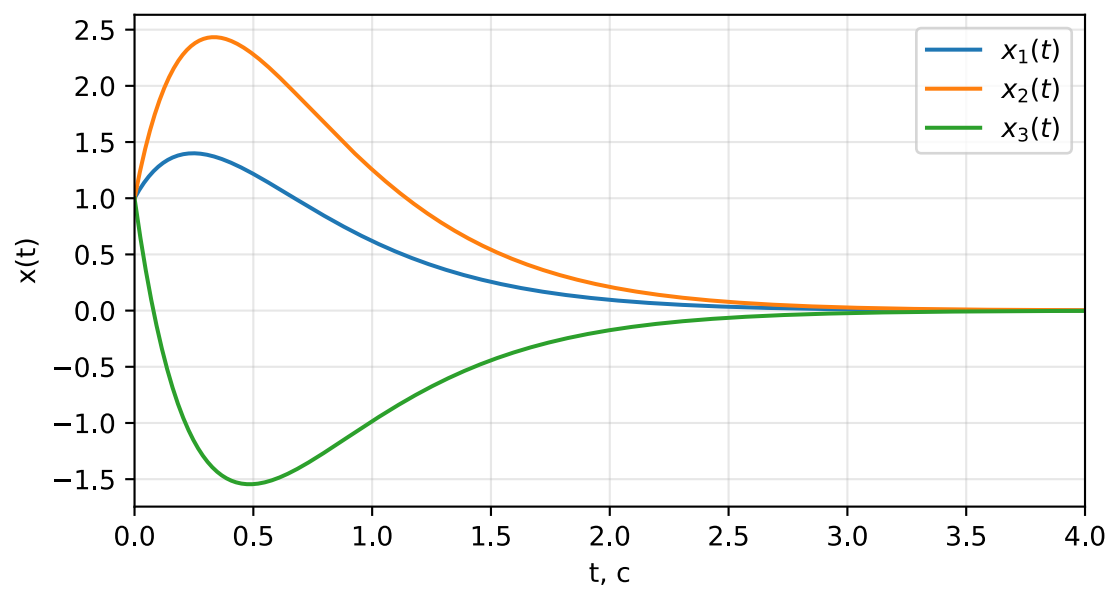


Рисунок 39. Компоненты состояния $x(t)$ для $(Q, R) = (I, 0)$

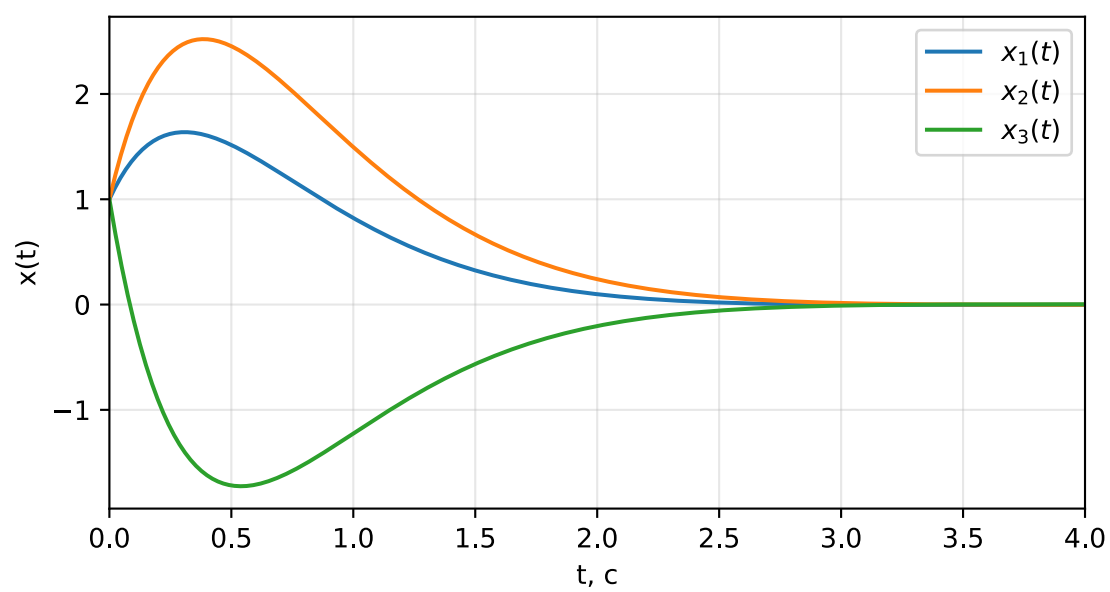


Рисунок 40. Компоненты состояния $x(t)$ для $(Q, R) = (0, 1)$

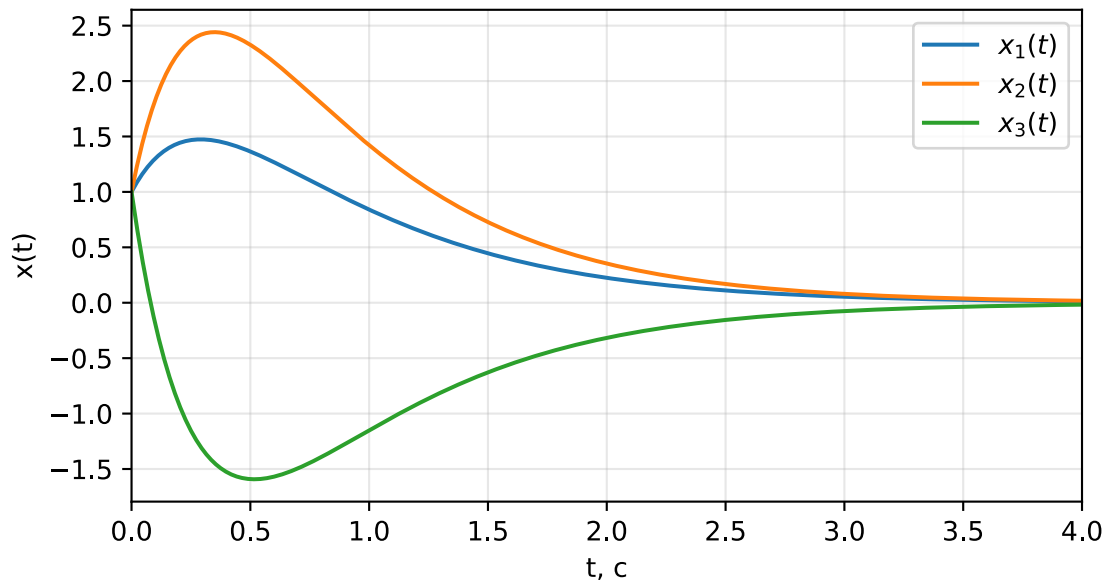


Рисунок 41. Компоненты состояния $x(t)$ для $(Q, R) = (0, 0)$

По моделированию:

- все четыре регулятора обеспечивают затухание состояния в соответствии с заданной круговой областью;
- варианты $(Q, R) = (0, 1)$ и $(0, 0)$ являются граничными: управляемые моды располагаются на самой границе круга, поэтому устойчивость сохраняется, но дополнительного запаса по расположению спектра уже нет.

Вывод

В Задании 3 показано, что подход с качественной экспоненциальной устойчивостью позволяет задавать не отдельные полюса, а геометрическую область их расположения на комплексной плоскости через параметры (β, r) . Для выбранных параметров $\beta = -2.5, r = 1$ корректный синтез получен для всех четырех наборов (Q, R) .

При $Q = I$ собственные числа располагаются строго внутри круга и создают запас по качественной экспоненциальной устойчивости. При $Q = 0$ решения становятся граничными: управляемые моды лежат на самой границе допустимой области. Для случая $(Q, R) = (0, 0)$ нетривиальное решение корректно восстанавливается только после перехода к управляемой части системы.

Общий вывод

Лабораторная работа показала, что немодальный синтез дает практичный инструмент управления, когда важна не точная расстановка полюсов, а выполнение требований к области их расположения и к скорости затухания.

На всех заданиях подтвердился общий компромисс: увеличение требуемой скорости (через α , а также через параметры (β, r)) обычно достигается ценой роста коэффициентов регулятора/наблюдателя и более жесткого управления. При этом корректный выбор постановки и параметров весов позволяет управлять этим компромиссом и получать не просто устойчивую, а практически реализуемую замкнутую систему.

Приложение

Код для задания 1

```
1  import json
2  import os
3  import warnings
4  from pathlib import Path
5
6  os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")
7
8  import control as ct
9  import cvxpy as cp
10 import matplotlib.pyplot as plt
11 import numpy as np
12
13 warnings.filterwarnings("ignore", category=UserWarning, module="cvxpy")
14
15 BASE_DIR = Path(__file__).resolve().parents[1]
16 PLOTS_DIR = BASE_DIR / "plots_task1"
17 RESULTS_PATH = Path(__file__).with_name("task1_results.json")
18
19 A = np.array([
20     [4.0, 1.0, 6.0],
21     [2.0, 0.0, 2.0],
22     [-2.0, -2.0, -4.0],
23 ])
24 B = np.array([[2.0], [-2.0], [2.0]])
25 x0 = np.ones(3)
26 T = np.linspace(0.0, 6.0, 1201)
27 COLORS = {
28     "K1": "tab:blue",
29     "K2": "tab:orange",
30     "K3": "tab:green",
31     "K4": "tab:red",
32     "x1": "tab:blue",
33     "x2": "tab:orange",
34     "x3": "tab:green",
35 }
36 ALPHAS = {"alpha1": 1.0, "alpha2": 2.0}
37
38
39 def as_real_list(values):
40     arr = np.asarray(values)
41     if np.iscomplexobj(arr):
42         out = []
```

```

43     for v in arr.flatten():
44         if abs(v.imag) < 1e-9:
45             out.append(float(np.real(v)))
46         else:
47             out.append({"re": float(np.real(v)), "im": float(np.imag(v))})
48     return out
49     return [float(v) for v in arr.flatten()]
50
51
52 def round_matrix(mat, digits=6):
53     return np.round(np.asarray(mat, dtype=float), digits).tolist()
54
55
56 def serialize_matrix(mat, digits=6):
57     arr = np.asarray(mat)
58     out = []
59     for row in arr:
60         out_row = []
61         for value in row:
62             if abs(np.imag(value)) < 1e-9:
63                 out_row.append(round(float(np.real(value)), digits))
64             else:
65                 out_row.append(
66                     {
67                         "re": round(float(np.real(value)), digits),
68                         "im": round(float(np.imag(value)), digits),
69                     }
70                 )
71         out.append(out_row)
72     return out
73
74
75 def solve_lmi(alpha, minimize):
76     n = A.shape[0]
77     m = B.shape[1]
78     P = cp.Variable((n, n), symmetric=True)
79     Y = cp.Variable((m, n))
80     lmi = A @ P + P @ A.T + 2.0 * alpha * P + B @ Y + Y.T @ B.T
81     constraints = [P >> np.eye(n), lmi << 0]
82     if minimize:
83         objective = cp.Minimize(cp.norm(Y, "fro"))
84     else:
85         objective = cp.Minimize(0)
86     problem = cp.Problem(objective, constraints)
87     problem.solve(solver=cp.SCS, eps=1e-7, max_iters=200000, verbose=False)
88     K = Y.value @ np.linalg.inv(P.value)

```

```

89     eig_cl = np.linalg.eigvals(A + B @ K)
90     return {
91         "status": problem.status,
92         "objective": None if not minimize else float(problem.value),
93         "P": P.value,
94         "Y": Y.value,
95         "K": K,
96         "eig_cl": eig_cl,
97     }
98
99
100 def solve_riccati(alpha, q_matrix):
101     a_shift = A + alpha * np.eye(A.shape[0])
102     b_shift = np.sqrt(2.0) * B
103     P, _, _ = ct.care(a_shift, b_shift, q_matrix, np.array([[1.0]]))
104     K = -(B.T @ P)
105     eig_cl = np.linalg.eigvals(A + B @ K)
106     return {"P": P, "K": K, "eig_cl": eig_cl}
107
108
109 def simulate(K):
110     acl = A + B @ K
111     sys = ct.ss(acl, np.zeros((3, 1)), np.eye(3), np.zeros((3, 1)))
112     response = ct.initial_response(sys, T=T, X0=x0, return_x=True)
113     t = np.asarray(response[0], dtype=float)
114     x = np.asarray(response[2], dtype=float)
115     u = (K @ x).reshape(-1)
116     return t, x, u
117
118
119 def save_plot(path, t=None):
120     ax = plt.gca()
121     if t is not None:
122         ax.set_xlim(float(t[0]), float(t[-1]))
123         ax.margins(x=0)
124     plt.grid(True, alpha=0.3)
125     plt.legend()
126     plt.tight_layout()
127     plt.savefig(path, format="svg")
128     plt.close()
129
130
131 def build_plots(results):
132     for alpha_key, alpha_data in results.items():
133         series = {}
134         for regulator_name in ["K1", "K2", "K3", "K4"]:

```

```

135         K = np.array(alpha_data[regulator_name]["K"])
136         series[regulator_name] = simulate(K)
137
138     plt.figure(figsize=(6.0, 3.3))
139     for regulator_name in ["K1", "K2", "K3", "K4"]:
140         t, _, u = series[regulator_name]
141         plt.plot(t, u, label=r"${regulator_name}(t)$",
142                 color=COLORS[regulator_name])
143     plt.xlabel("t, c")
144     plt.ylabel("u(t)")
145     save_plot(PLOTS_DIR / f"task1_{alpha_key}_u_all.svg", t=t)
146
147     for regulator_name in ["K1", "K2", "K3", "K4"]:
148         t, x, _ = series[regulator_name]
149         plt.figure(figsize=(6.0, 3.3))
150         plt.plot(t, x[0], label=r"$x_1(t)$", color=COLORS["x1"])
151         plt.plot(t, x[1], label=r"$x_2(t)$", color=COLORS["x2"])
152         plt.plot(t, x[2], label=r"$x_3(t)$", color=COLORS["x3"])
153         plt.xlabel("t, c")
154         plt.ylabel("x(t)")
155         save_plot(PLOTS_DIR / f"task1_{alpha_key}_x_{regulator_name}.svg",
156                 t=t)
157
158     alpha_data[regulator_name]["u0"] = float(series[regulator_name][2]
159                                             [0])
160     alpha_data[regulator_name]["max_u_abs"] =
161     float(np.max(np.abs(series[regulator_name][2])))
162     alpha_data[regulator_name]["max_x_abs"] =
163     float(np.max(np.abs(series[regulator_name][1])))
164
165
166 def main():
167     eig_A = np.linalg.eigvals(A)
168     AB = A @ B
169     A2B = A @ AB
170     U = ct.ctrb(A, B)
171     hautus = {}
172     for eig in eig_A:
173         H = np.hstack((eig * np.eye(3) - A, B))
174         hautus_key = f"{eig.real:.6f}{eig.imag:+.6f}j"
175         hautus[hautus_key] = {
176             "lambda": {"re": float(eig.real), "im": float(eig.imag)},
177             "H": serialize_matrix(H),
178             "rank": int(np.linalg.matrix_rank(H)),
179         }
180     results = {}

```

```

177     for alpha_key, alpha in ALPHAS.items():
178         lmi_free = solve_lmi(alpha, minimize=False)
179         lmi_min = solve_lmi(alpha, minimize=True)
180         riccati_qi = solve_riccati(alpha, np.eye(3))
181         riccati_q0 = solve_riccati(alpha, np.zeros((3, 3)))
182         results[alpha_key] = {
183             "alpha": alpha,
184             "K1": {
185                 "status": lmi_free["status"],
186                 "P": round_matrix(lmi_free["P"]),
187                 "Y": round_matrix(lmi_free["Y"]),
188                 "K": round_matrix(lmi_free["K"]),
189                 "eig_cl": as_real_list(np.round(lmi_free["eig_cl"], 6)),
190             },
191             "K2": {
192                 "status": lmi_min["status"],
193                 "objective": float(lmi_min["objective"]),
194                 "P": round_matrix(lmi_min["P"]),
195                 "Y": round_matrix(lmi_min["Y"]),
196                 "K": round_matrix(lmi_min["K"]),
197                 "eig_cl": as_real_list(np.round(lmi_min["eig_cl"], 6)),
198             },
199             "K3": {
200                 "P": round_matrix(riccati_qi["P"]),
201                 "K": round_matrix(riccati_qi["K"]),
202                 "eig_cl": as_real_list(np.round(riccati_qi["eig_cl"], 6)),
203             },
204             "K4": {
205                 "P": round_matrix(riccati_q0["P"]),
206                 "K": round_matrix(riccati_q0["K"]),
207                 "eig_cl": as_real_list(np.round(riccati_q0["eig_cl"], 6)),
208             },
209         }
210
211     build_plots(results)
212
213     payload = {
214         "A": round_matrix(A),
215         "B": round_matrix(B),
216         "AB": round_matrix(AB),
217         "A2B": round_matrix(A2B),
218         "eig_A": as_real_list(np.round(eig_A, 6)),
219         "U": round_matrix(U),
220         "rank_U": int(np.linalg.matrix_rank(U)),
221         "det_U": float(np.linalg.det(U)),
222         "alpha_max": 2.0,

```

```

223     "hautus": hautus,
224     "alphas": results,
225 }
226 RESULTS_PATH.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
227                           encoding="utf-8")
228
229 if __name__ == "__main__":
230     main()

```

Код для задания 2

```

1  import json
2  import os
3  import warnings
4  from pathlib import Path
5
6  os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")
7
8  import control as ct
9  import cvxpy as cp
10 import matplotlib.pyplot as plt
11 import numpy as np
12
13 warnings.filterwarnings("ignore", category=UserWarning, module="cvxpy")
14
15 BASE_DIR = Path(__file__).resolve().parents[1]
16 PLOTS_DIR = BASE_DIR / "plots_task2"
17 RESULTS_PATH = Path(__file__).with_name("task2_results.json")
18
19 A = np.array([
20     [6.0, 0.0, -12.0, 6.0],
21     [0.0, 6.0, -6.0, 12.0],
22     [-12.0, -6.0, 6.0, 0.0],
23     [6.0, 12.0, 0.0, 6.0],
24 ])
25 B = np.array([[6.0], [12.0], [6.0], [4.0]])
26 C = np.array([
27     [-6.0, 6.0, 6.0, 6.0],
28     [3.0, 0.0, 0.0, 3.0],
29 ])
30
31 X0 = np.ones(4)
32 XHAT0 = np.zeros(4)
33 T = np.linspace(0.0, 1.0, 1001)
34

```



```

35 ALPHA_SET = [8.0, 12.0]
36 PAIRS = {
37     "pair_eq": {"alpha_k": 8.0, "alpha_l": 8.0, "label": r"$\\alpha_K=8,\\;\\alpha_L=8$"},
38     "pair_k_gt_l": {"alpha_k": 12.0, "alpha_l": 8.0, "label": r"$\\alpha_K=12,\\;\\alpha_L=8$"},
39     "pair_k_lt_l": {"alpha_k": 8.0, "alpha_l": 12.0, "label": r"$\\alpha_K=8,\\;\\alpha_L=12$"},
40 }
41
42 COLORS = {
43     "x1": "tab:blue",
44     "x2": "tab:orange",
45     "x3": "tab:green",
46     "x4": "tab:red",
47     "hat": "cyan",
48 }
49
50
51 def round_matrix(mat, digits=6):
52     return np.round(np.asarray(mat, dtype=float), digits).tolist()
53
54
55 def as_real_list(values):
56     arr = np.asarray(values)
57     if np.iscomplexobj(arr):
58         out = []
59         for v in arr.flatten():
60             if abs(v.imag) < 1e-9:
61                 out.append(float(np.real(v)))
62             else:
63                 out.append({"re": float(np.real(v)), "im": float(np.imag(v))})
64     return out
65     return [float(v) for v in arr.flatten()]
66
67
68 def serialize_matrix(mat, digits=6):
69     arr = np.asarray(mat)
70     out = []
71     for row in arr:
72         out_row = []
73         for value in row:
74             if abs(np.imag(value)) < 1e-9:
75                 out_row.append(round(float(np.real(value)), digits))
76             else:
77                 out_row.append(

```

```

78         {
79             "re": round(float(np.real(value)), digits),
80             "im": round(float(np.imag(value)), digits),
81         }
82     )
83     out.append(out_row)
84     return out
85
86
87 def solve_lmi_k(alpha):
88     n = A.shape[0]
89     m = B.shape[1]
90     P = cp.Variable((n, n), symmetric=True)
91     Y = cp.Variable((m, n))
92     lmi = A @ P + P @ A.T + 2.0 * alpha * P + B @ Y + Y.T @ B.T
93     constraints = [P >> np.eye(n), lmi << 0]
94     problem = cp.Problem(cp.Minimize(0), constraints)
95     problem.solve(solver=cp.SCS, eps=1e-7, max_iters=200000, verbose=False)
96     K = Y.value @ np.linalg.inv(P.value)
97     eig_cl = np.linalg.eigvals(A + B @ K)
98     return {
99         "status": problem.status,
100        "objective": float(problem.value),
101        "P": P.value,
102        "Y": Y.value,
103        "K": K,
104        "eig": eig_cl,
105    }
106
107
108 def solve_lmi_l(alpha):
109     n = A.shape[0]
110     p = C.shape[0]
111     Ad = A.T
112     Bd = C.T
113     P = cp.Variable((n, n), symmetric=True)
114     Y = cp.Variable((p, n))
115     lmi = Ad @ P + P @ Ad.T + 2.0 * alpha * P + Bd @ Y + Y.T @ Bd.T
116     constraints = [P >> np.eye(n), lmi << 0]
117     problem = cp.Problem(cp.Minimize(cp.norm(Y, "fro")), constraints)
118     problem.solve(solver=cp.SCS, eps=1e-7, max_iters=200000, verbose=False)
119     Kd = Y.value @ np.linalg.inv(P.value)
120     L = Kd.T
121     eig_obs = np.linalg.eigvals(A + L @ C)
122     return {
123         "status": problem.status,

```

```

124         "objective": float(problem.value),
125         "P": P.value,
126         "Y": Y.value,
127         "L": L,
128         "eig": eig_obs,
129     }
130
131
132 def simulate_pair(K, L):
133     Aug = np.block([
134         [A, B @ K],
135         [-L @ C, A + B @ K + L @ C],
136     ])
137     Caug = np.eye(8)
138     sys = ct.ss(Aug, np.zeros((8, 1)), Caug, np.zeros((8, 1)))
139     x0_aug = np.concatenate([X0, XHAT0])
140     response = ct.initial_response(sys, T=T, X0=x0_aug, return_x=True)
141     t = np.asarray(response[0], dtype=float)
142     x_aug = np.asarray(response[2], dtype=float)
143     x = x_aug[:4]
144     xhat = x_aug[4:]
145     e = x - xhat
146     u = (K @ xhat).reshape(-1)
147     return t, x, xhat, e, u
148
149
150 def save_plot(path, t=None):
151     ax = plt.gca()
152     if t is not None:
153         ax.set_xlim(float(t[0]), float(t[-1]))
154         ax.margins(x=0)
155     plt.grid(True, alpha=0.3)
156     plt.legend()
157     plt.tight_layout()
158     plt.savefig(path, format="svg")
159     plt.close()
160
161
162 def plot_pair(pair_key, pair_label, t, x, xhat, e, u):
163     plt.figure(figsize=(6.0, 3.3))
164     plt.plot(t, u, label=r"$u(t)$", color="tab:red")
165     plt.xlabel("t, c")
166     plt.ylabel("u(t)")
167     save_plot(PLOTS_DIR / f"task2_{pair_key}_u.svg", t=t)
168
169     for idx, cname in enumerate(["x1", "x2", "x3", "x4"]):

```

```

170     plt.figure(figsize=(6.0, 3.3))
171     plt.plot(t, x[idx], label=r"$x_{idx+1}(t)$", color=COLORS[cname])
172     plt.plot(t, xhat[idx], label=r"$\hat{x}_{idx+1}(t)$",
173              color=COLORS["hat"], linestyle="--")
174     plt.xlabel("t, c")
175     plt.ylabel(f"x_{idx+1}(t)")
176     save_plot(PLOTS_DIR / f"task2_{pair_key}_compare_x{idx+1}.svg", t=t)
177
178     plt.figure(figsize=(6.0, 3.3))
179     plt.plot(t, e[0], label=r"$e_1(t)$", color=COLORS["x1"])
180     plt.plot(t, e[1], label=r"$e_2(t)$", color=COLORS["x2"])
181     plt.plot(t, e[2], label=r"$e_3(t)$", color=COLORS["x3"])
182     plt.plot(t, e[3], label=r"$e_4(t)$", color=COLORS["x4"])
183     plt.xlabel("t, c")
184     plt.ylabel("e(t)")
185     save_plot(PLOTS_DIR / f"task2_{pair_key}_error.svg", t=t)
186
187 def main():
188     PLOTS_DIR.mkdir(parents=True, exist_ok=True)
189
190     eig_A = np.linalg.eigvals(A)
191     U = ct.ctrb(A, B)
192     V = ct.obsv(A, C)
193
194     AB = A @ B
195     A2B = A @ AB
196     A3B = A @ A2B
197
198     CA = C @ A
199     CA2 = C @ A @ A
200     CA3 = C @ A @ A @ A
201
202     hautus_ctrl = {}
203     hautus_obs = {}
204     for eig in eig_A:
205         Hc = np.hstack((eig * np.eye(4) - A, B))
206         Ho = np.vstack((eig * np.eye(4) - A, C))
207         key = f"{eig.real:.6f}{eig.imag:+.6f}j"
208         hautus_ctrl[key] = {
209             "lambda": {"re": float(eig.real), "im": float(eig.imag)},
210             "Hc": serialize_matrix(Hc),
211             "rank": int(np.linalg.matrix_rank(Hc)),
212         }
213         hautus_obs[key] = {
214             "lambda": {"re": float(eig.real), "im": float(eig.imag)},

```

```

215         "Ho": serialize_matrix(Ho),
216         "rank": int(np.linalg.matrix_rank(Ho)),
217     }
218
219     lmi_k = {alpha: solve_lmi_k(alpha) for alpha in ALPHA_SET}
220     lmi_l = {alpha: solve_lmi_l(alpha) for alpha in ALPHA_SET}
221
222     pair_results = {}
223     for pair_key, pair_cfg in PAIRS.items():
224         K = lmi_k[pair_cfg["alpha_k"]]["K"]
225         L = lmi_l[pair_cfg["alpha_l"]]["L"]
226         eig_k = np.linalg.eigvals(A + B @ K)
227         eig_l = np.linalg.eigvals(A + L @ C)
228         t, x, xhat, e, u = simulate_pair(K, L)
229         plot_pair(pair_key, pair_cfg["label"], t, x, xhat, e, u)
230
231         pair_results[pair_key] = {
232             "alpha_k": pair_cfg["alpha_k"],
233             "alpha_l": pair_cfg["alpha_l"],
234             "K": round_matrix(K),
235             "L": round_matrix(L),
236             "eig_A_BK": as_real_list(np.round(eig_k, 6)),
237             "eig_A_LC": as_real_list(np.round(eig_l, 6)),
238             "u0": float(u[0]),
239             "max_u_abs": float(np.max(np.abs(u))),
240             "max_e_abs": float(np.max(np.abs(e))),
241             "e_final_norm": float(np.linalg.norm(e[:, -1])),
242         }
243
244     payload = {
245         "A": round_matrix(A),
246         "B": round_matrix(B),
247         "C": round_matrix(C),
248         "eig_A": as_real_list(np.round(eig_A, 6)),
249         "AB": round_matrix(AB),
250         "A2B": round_matrix(A2B),
251         "A3B": round_matrix(A3B),
252         "CA": round_matrix(CA),
253         "CA2": round_matrix(CA2),
254         "CA3": round_matrix(CA3),
255         "U": round_matrix(U),
256         "V": round_matrix(V),
257         "rank_U": int(np.linalg.matrix_rank(U)),
258         "rank_V": int(np.linalg.matrix_rank(V)),
259         "det_U": float(np.linalg.det(U)),
260         "alpha_k_max": None,

```

```

261     "alpha_l_max": 12.0,
262     "hautus_ctrl": hautus_ctrl,
263     "hautus_obs": hautus_obs,
264     "lmi_k": {
265         str(alpha): {
266             "status": data["status"],
267             "objective": data["objective"],
268             "P": round_matrix(data["P"]),
269             "Y": round_matrix(data["Y"]),
270             "K": round_matrix(data["K"]),
271             "eig": as_real_list(np.round(data["eig"], 6)),
272         }
273         for alpha, data in lmi_k.items()
274     },
275     "lmi_l": {
276         str(alpha): {
277             "status": data["status"],
278             "objective": data["objective"],
279             "P": round_matrix(data["P"]),
280             "Y": round_matrix(data["Y"]),
281             "L": round_matrix(data["L"]),
282             "eig": as_real_list(np.round(data["eig"], 6)),
283         }
284         for alpha, data in lmi_l.items()
285     },
286     "pairs": pair_results,
287 }
288
289 RESULTS_PATH.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
290                           encoding="utf-8")
291
292 if __name__ == "__main__":
293     main()

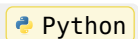
```

Код для задания 3

```

1  import json
2  import os
3  from pathlib import Path
4
5  os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")
6
7  import control as ct
8  import matplotlib.pyplot as plt
9  import numpy as np

```



```

10 from scipy.optimize import root
11
12 BASE_DIR = Path(__file__).resolve().parents[1]
13 PLOTS_DIR = BASE_DIR / "plots_task3"
14 RESULTS_PATH = Path(__file__).with_name("task3_results.json")
15
16 A = np.array([
17     [4.0, 1.0, 6.0],
18     [2.0, 0.0, 2.0],
19     [-2.0, -2.0, -4.0],
20 ])
21 B = np.array([[2.0], [-2.0], [2.0]])
22
23 T_CTRL = np.array([
24     [2.0, 18.0, -1.0],
25     [-2.0, 8.0, 0.0],
26     [2.0, -8.0, 1.0],
27 ])
28 T_CTRL_INV = np.array([
29     [2.0 / 13.0, -5.0 / 26.0, 2.0 / 13.0],
30     [1.0 / 26.0, 1.0 / 13.0, 1.0 / 26.0],
31     [0.0, 1.0, 1.0],
32 ])
33
34 BETA = -2.5
35 RADIUS = 1.0
36
37 X0 = np.ones(3)
38 T = np.linspace(0.0, 4.0, 1601)
39
40 CASES = {
41     "Q1_R1": {"Q": np.eye(3), "R": 1.0, "label": "Q=I, R=1", "color":
42         "tab:blue"},
43     "Q1_R0": {"Q": np.eye(3), "R": 0.0, "label": "Q=I, R=0", "color":
44         "tab:orange"},
45     "Q0_R1": {"Q": np.zeros((3, 3)), "R": 1.0, "label": "Q=0, R=1", "color":
46         "tab:green"},
47     "Q0_R0": {"Q": np.zeros((3, 3)), "R": 0.0, "label": "Q=0, R=0", "color":
48         "tab:red"},
49 }
50
51 np.random.seed(15)
52
53 def round_matrix(mat, digits=6):
54     return np.round(np.asarray(mat, dtype=float), digits).tolist()

```

```

52
53
54 def serialize_values(values, digits=6):
55     arr = np.asarray(values)
56     out = []
57     for v in arr.flatten():
58         if abs(np.imag(v)) < 1e-9:
59             out.append(round(float(np.real(v)), digits))
60         else:
61             out.append(
62                 {
63                     "re": round(float(np.real(v)), digits),
64                     "im": round(float(np.imag(v)), digits),
65                 }
66             )
67     return out
68
69
70 def unpack_p(x):
71     p11, p12, p13, p22, p23, p33 = x
72     return np.array(
73         [
74             [p11, p12, p13],
75             [p12, p22, p23],
76             [p13, p23, p33],
77         ]
78     )
79
80
81 def solve_case(Q, R, tries=120):
82     A0 = A - BETA * np.eye(3)
83
84     def residual(x):
85         P = unpack_p(x)
86         m = R + float((B.T @ P @ B).item())
87         if abs(m) < 1e-10:
88             return np.ones(6) * 1e6
89         K = -(1.0 / m) * (B.T @ P @ A0)
90         F = A + B @ K - BETA * np.eye(3)
91         S = F.T @ P @ F - (RADIUS**2) * P + Q
92         return np.array([S[0, 0], S[0, 1], S[0, 2], S[1, 1], S[1, 2], S[2, 2]])
93
94     best = None
95     scales = [1e-3, 1e-2, 1e-1, 1.0, 10.0, 40.0]
96
97     for scale in scales:

```



```

98         for _ in range(tries // len(scales)):
99             x0 = np.array([1, 0, 0, 1, 0, 1], dtype=float) * scale
100             x0 += np.random.randn(6) * 0.3 * scale
101             sol = root(residual, x0, method="hybr")
102             if not sol.success:
103                 continue
104             res_norm = float(np.linalg.norm(residual(sol.x)))
105
106             P = unpack_p(sol.x)
107             m = R + float((B.T @ P @ B).item())
108             if abs(m) < 1e-9:
109                 continue
110
111             K = -(1.0 / m) * (B.T @ P @ A0)
112             eig_cl = np.linalg.eigvals(A + B @ K)
113             max_dist = float(np.max(np.abs(eig_cl - BETA)))
114
115             candidate = {
116                 "residual": res_norm,
117                 "P": P,
118                 "K": K,
119                 "eig_cl": eig_cl,
120                 "max_dist": max_dist,
121                 "inside_circle": max_dist <= RADIUS + 1e-6,
122             }
123
124             if best is None:
125                 best = candidate
126             else:
127                 better = False
128                 if candidate["inside_circle"] and not best["inside_circle"]:
129                     better = True
130                 elif candidate["inside_circle"] == best["inside_circle"]:
131                     if candidate["max_dist"] < best["max_dist"] - 1e-9:
132                         better = True
133                     elif abs(candidate["max_dist"] - best["max_dist"]) < 1e-9
134                         and candidate["residual"] < best["residual"]:
135                         better = True
136                 if better:
137                     best = candidate
138
139             return best
140
141 def fallback_q0_r0():
142     A_tilde = T_CTRL_INV @ A @ T_CTRL

```

```

143     B_tilde = T_CTRL_INV @ B
144     A_c = A_tilde[:2, :2]
145     B_c = B_tilde[:2, :]
146
147     # For Q=0, R=0 choose the controlled poles on the boundary of the target
    circle.
148     poles_c = [BETA - RADIUS, BETA + RADIUS]
149     K_c = -ct.place(A_c, B_c, poles_c)
150     K_tilde = np.hstack((K_c, np.zeros((1, 1))))
151     K = K_tilde @ T_CTRL_INV
152
153     F_c = A_c + B_c @ K_c - BETA * np.eye(2)
154     basis = [
155         np.array([[1.0, 0.0], [0.0, 0.0]]),
156         np.array([[0.0, 1.0], [1.0, 0.0]]),
157         np.array([[0.0, 0.0], [0.0, 1.0]]),
158     ]
159     rows = []
160     for mat in basis:
161         s_mat = F_c.T @ mat @ F_c - (RADIUS**2) * mat
162         rows.append([s_mat[0, 0], s_mat[0, 1], s_mat[1, 1]])
163     mtx = np.array(rows, dtype=float).T
164     _, _, vh = np.linalg.svd(mtx)
165     coeffs = vh[-1]
166     P_c = coeffs[0] * basis[0] + coeffs[1] * basis[1] + coeffs[2] * basis[2]
167     if abs((B_c.T @ P_c @ B_c).item()) < 1e-9:
168         P_c = -P_c
169
170     P_tilde = np.zeros((3, 3))
171     P_tilde[:2, :2] = P_c
172     P = T_CTRL_INV.T @ P_tilde @ T_CTRL_INV
173
174     eig_cl = np.linalg.eigvals(A + B @ K)
175     max_dist = float(np.max(np.abs(eig_cl - BETA)))
176     return {
177         "residual": float(np.linalg.norm((A + B @ K - BETA * np.eye(3)).T @ P @
    (A + B @ K - BETA * np.eye(3)) - (RADIUS**2) * P)),
178         "P": P,
179         "K": K,
180         "eig_cl": eig_cl,
181         "max_dist": max_dist,
182         "inside_circle": max_dist <= RADIUS + 1e-6,
183     }
184
185
186 def simulate_closed_loop(K):

```

```

187     Acl = A + B @ K
188     sys = ct.ss(Acl, np.zeros((3, 1)), np.eye(3), np.zeros((3, 1)))
189     response = ct.initial_response(sys, T=T, X0=X0, return_x=True)
190     t = np.asarray(response[0], dtype=float)
191     x = np.asarray(response[2], dtype=float)
192     u = (K @ x).reshape(-1)
193     return t, x, u
194
195
196 def save_plot(path, t=None):
197     ax = plt.gca()
198     if t is not None:
199         ax.set_xlim(float(t[0]), float(t[-1]))
200         ax.margins(x=0)
201     plt.grid(True, alpha=0.3)
202     plt.legend()
203     plt.tight_layout()
204     plt.savefig(path, format="svg")
205     plt.close()
206
207
208 def build_plots(case_results):
209     theta = np.linspace(0.0, 2.0 * np.pi, 601)
210
211     plt.figure(figsize=(6.2, 4.8))
212     circle_x = BETA + RADIUS * np.cos(theta)
213     circle_y = RADIUS * np.sin(theta)
214     plt.plot(circle_x, circle_y, color="black", linestyle="--", linewidth=1.4,
215             label="Граница круга (beta, r)")
216
217     for key, cfg in CASES.items():
218         eig = np.array(case_results[key]["eig_cl_complex"], dtype=complex)
219         plt.scatter(eig.real, eig.imag, color=cfg["color"], s=34,
220                 label=cfg["label"])
221
222     plt.axhline(0.0, color="0.4", linewidth=0.8)
223     plt.axvline(0.0, color="0.4", linewidth=0.8)
224     plt.xlabel("Re")
225     plt.ylabel("Im")
226     plt.grid(True, alpha=0.3)
227     plt.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=3,
228             frameon=True)
229     plt.tight_layout(rect=(0.0, 0.08, 1.0, 1.0))
230     plt.savefig(PLOTS_DIR / "task3_eigs_circle.svg", format="svg")
231     plt.close()

```

```

230     plt.figure(figsize=(6.0, 3.3))
231     for key, cfg in CASES.items():
232         t = np.array(case_results[key]["t"])
233         u = np.array(case_results[key]["u"])
234         if key == "Q1_R0":
235             linestyle = "--"
236         elif key == "Q0_R0":
237             linestyle = "-."
238         else:
239             linestyle = "-"
240         plt.plot(t, u, color=cfg["color"], linestyle=linestyle,
241                 label=cfg["label"])
242     plt.xlabel("t, c")
243     plt.ylabel("u(t)")
244     save_plot(PLOTS_DIR / "task3_u_all.svg", t=T)
245
246     for key in CASES:
247         t = np.array(case_results[key]["t"])
248         x = np.array(case_results[key]["x"])
249         plt.figure(figsize=(6.0, 3.3))
250         plt.plot(t, x[0], label=r"$x_1(t)$", color="tab:blue")
251         plt.plot(t, x[1], label=r"$x_2(t)$", color="tab:orange")
252         plt.plot(t, x[2], label=r"$x_3(t)$", color="tab:green")
253         plt.xlabel("t, c")
254         plt.ylabel("x(t)")
255         save_plot(PLOTS_DIR / f"task3_x_{key}.svg", t=t)
256
257 def main():
258     PLOTS_DIR.mkdir(parents=True, exist_ok=True)
259
260     eig_A = np.linalg.eigvals(A)
261     U = ct.ctrb(A, B)
262
263     case_results = {}
264     for key, cfg in CASES.items():
265         solved = solve_case(cfg["Q"], cfg["R"])
266         if key == "Q0_R0" and (solved is None or not solved["inside_circle"]):
267             solved = fallback_q0_r0()
268         if solved is None:
269             K = np.zeros((1, 3))
270             P = np.eye(3)
271             eig_cl = np.linalg.eigvals(A + B @ K)
272             t, x, u = simulate_closed_loop(K)
273             case_results[key] = {
274                 "Q": round_matrix(cfg["Q"]),

```

```

275         "R": float(cfg["R"]),
276         "K": round_matrix(K),
277         "P": round_matrix(P),
278         "residual": None,
279         "inside_circle": False,
280         "max_dist": float(np.max(np.abs(eig_cl - BETA))),
281         "eig_cl": serialize_values(np.round(eig_cl, 6)),
282         "eig_cl_complex": eig_cl.tolist(),
283         "t": t.tolist(),
284         "x": x.tolist(),
285         "u": u.tolist(),
286         "u0": float(u[0]),
287         "max_u_abs": float(np.max(np.abs(u))),
288         "max_x_abs": float(np.max(np.abs(x))),
289     }
290     continue
291
292     K = solved["K"]
293     P = solved["P"]
294     eig_cl = solved["eig_cl"]
295     t, x, u = simulate_closed_loop(K)
296
297     case_results[key] = {
298         "Q": round_matrix(cfg["Q"]),
299         "R": float(cfg["R"]),
300         "K": round_matrix(K),
301         "P": round_matrix(P),
302         "residual": float(solved["residual"]),
303         "inside_circle": bool(solved["inside_circle"]),
304         "max_dist": float(solved["max_dist"]),
305         "eig_cl": serialize_values(np.round(eig_cl, 6)),
306         "eig_cl_complex": eig_cl.tolist(),
307         "t": t.tolist(),
308         "x": x.tolist(),
309         "u": u.tolist(),
310         "u0": float(u[0]),
311         "max_u_abs": float(np.max(np.abs(u))),
312         "max_x_abs": float(np.max(np.abs(x))),
313     }
314
315     build_plots(case_results)
316
317     cases_json = {}
318     for key, val in case_results.items():
319         cases_json[key] = {
320             **val,

```

```

321         "eig_cl_complex": serialize_values(np.array(val["eig_cl_complex"]),
322         digits=6),
323     }
324     payload = {
325         "A": round_matrix(A),
326         "B": round_matrix(B),
327         "eig_A": serialize_values(np.round(eig_A, 6)),
328         "U": round_matrix(U),
329         "rank_U": int(np.linalg.matrix_rank(U)),
330         "det_U": float(np.linalg.det(U)),
331         "beta": BETA,
332         "r": RADIUS,
333         "circle_center": [BETA, 0.0],
334         "cases": cases_json,
335     }
336
337     RESULTS_PATH.write_text(json.dumps(payload, ensure_ascii=False, indent=2),
338     encoding="utf-8")
339
340 if __name__ == "__main__":
341     main()

```